



数字逻辑与处理器基础

Fundamentals of Digital Logic and Processors



第六讲 微处理器设计

第2部分 从单周期到多周期、中断与异常

李学清

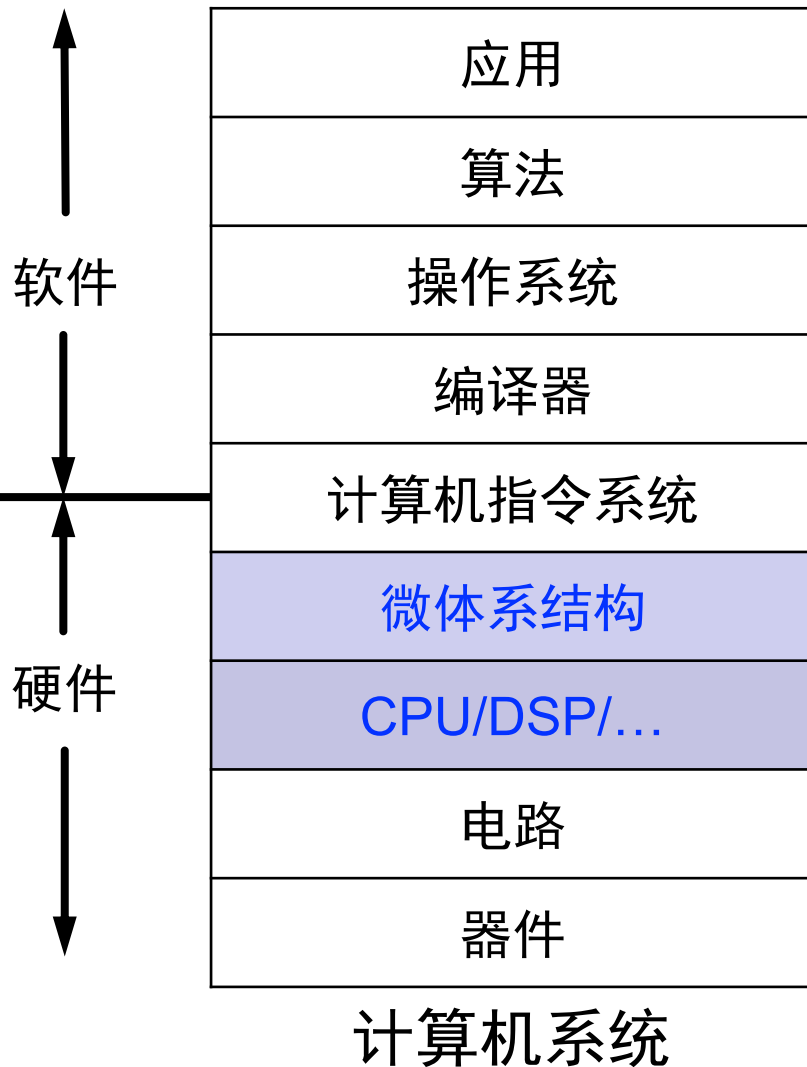
助教：陈仲豪*/唐文骏/曾令安

#腾讯会议：453-1488-6548

*本节内容负责答疑助教

2024年11月18日

微体系结构



周	日期	暂定主要内容
1	9/9	绪论
2	9/21	布尔代数 (中秋节调休)
3	9/23	组合逻辑
4	9/30	时序逻辑
5	10/7	国庆节放假
6	10/14	时序逻辑
7	10/21	计算机指令系统
8	11/3	期中考试
9	11/4	计算机指令系统
10	11/11	微处理器
11	11/18	微处理器
12	11/25	流水线
13	12/2	流水线
13	12/7	流水线/存储器 (调课)
15	12/16	存储器
16	12/23	存储器/IO与总线/总结

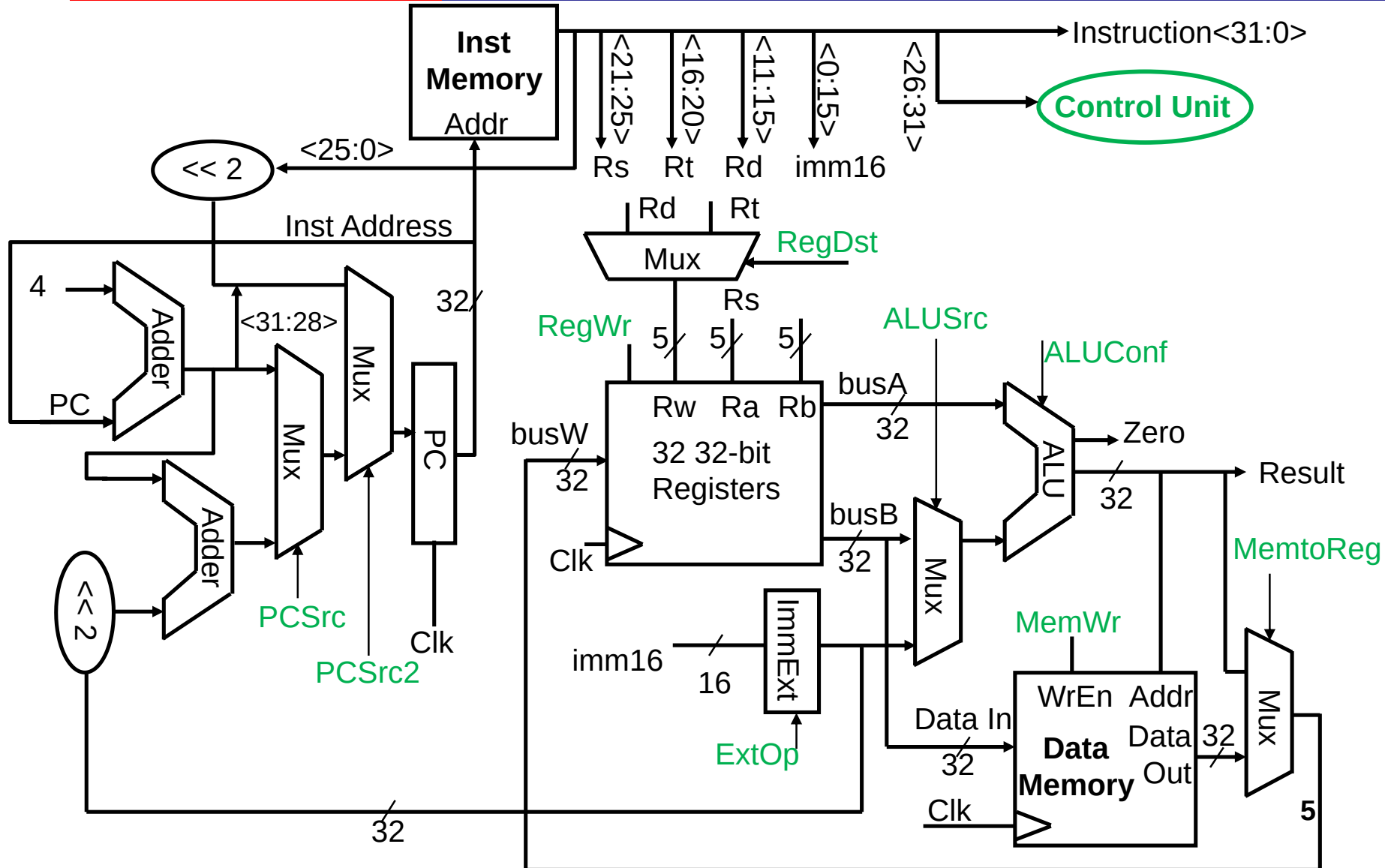
本节课学习目标

- 知识点：概念、原理和指标
 - 多周期处理器基本指令的执行过程
 - 多周期处理器的硬件模块
- 能力
 - 掌握设计和分析多周期处理器的方法
 - 掌握设计折衷和评估方法
- 思想
 - 同步和结构化计算
 - 折衷

本节课目录

- 复习与回顾
- 单周期数据通路设计流程
 - 指令集和设计需求分析
 - 功能模块设计
 - 数据通路模块组装
 - 控制信号分析与逻辑设计
- 单周期数据通路延时分析
- 多周期数据通路
- 异常与中断

从零开始：单周期数据通路



成功实现：单周期控制信号

See Appendix A

	func	10 0000	10 0010	We Don't Care :-)			
	op	00 0000	00 0000	00 1101	10 0011	10 1011	00 0100 00 0010
		add	sub	ori	lw	sw	beq jump
RegDst		1	1	0	0	x	x x
ALUSrc		0	0	1	1	1	0 x
MemtoReg		0	0	0	1	x	x x
RegWrite		1	1	1	1	0	0 0
MemWrite		0	0	0	0	1	0 0
PCSrc		0	0	0	0	0	ZERO x
PCSrc2		0	0	0	0	0	0 1
ExtOp		x	x	0	1	1	1 x
ALUConf<2:0>		Add	Subtract	Or	Add	Add	Subtract xxx

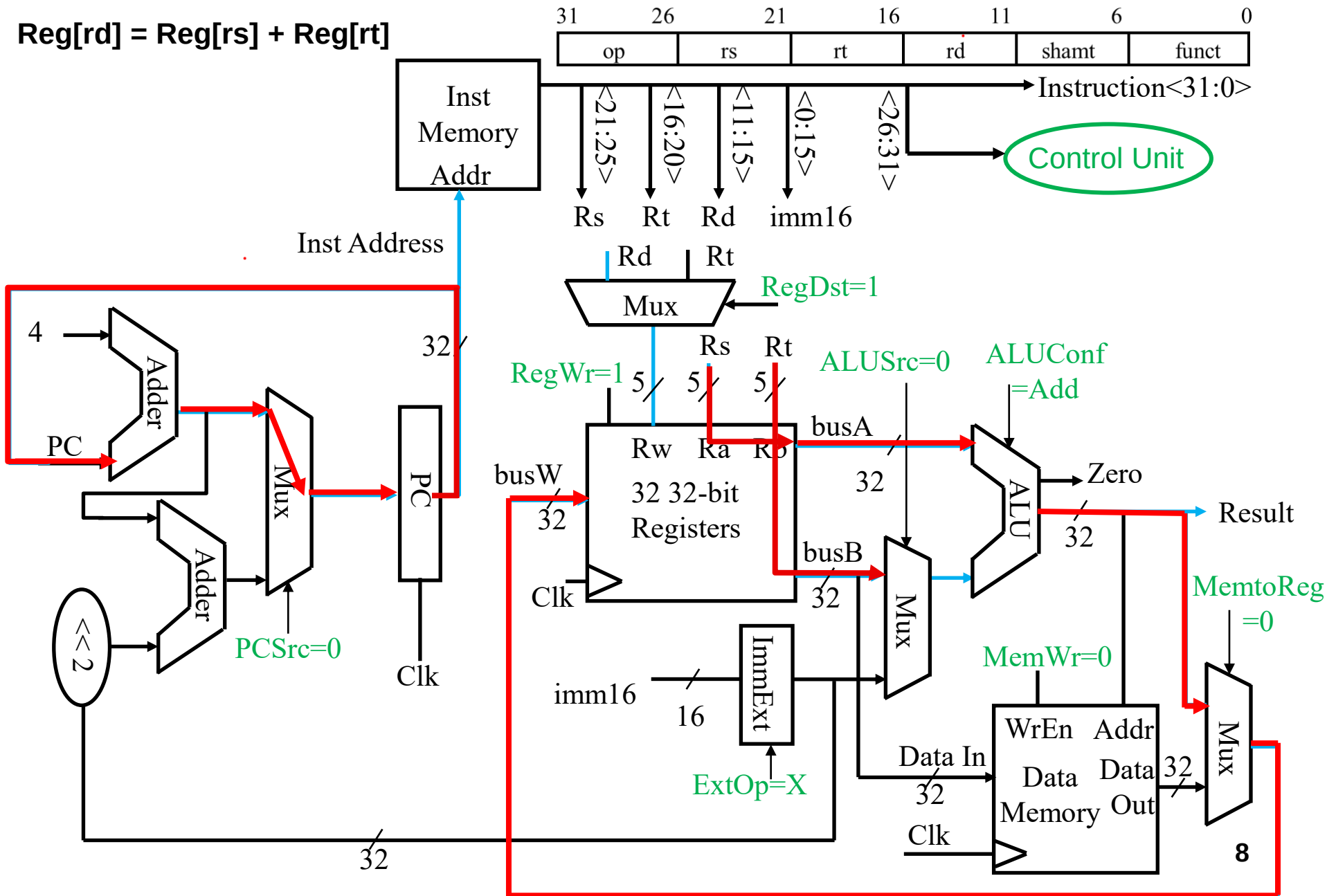
	31	26	21	16	11	6	0							
R-type	op		rs		rt		rd		shamt		funct		add, sub	
I-type	op		rs		rt		immediate						ori, lw, sw, beq	
J-type	op		target address										jump	6

做得好不好？

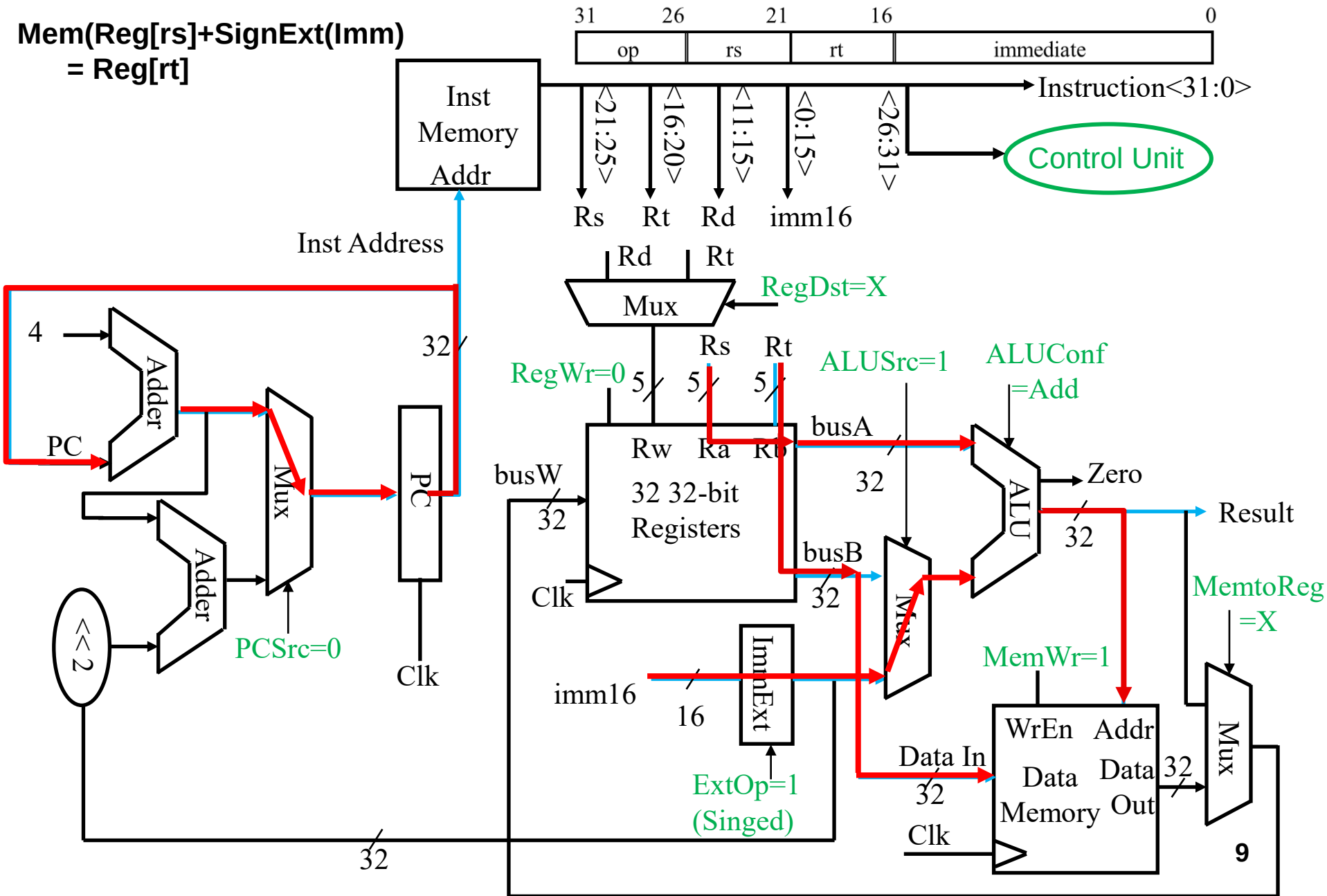
- 单周期数据通路 => $CPI=1$ (?!)
- 从性能角度看，单周期处理器快么？
 - 时钟周期由什么决定？
 - 性能极限在哪里？

Add: 单周期数据通路

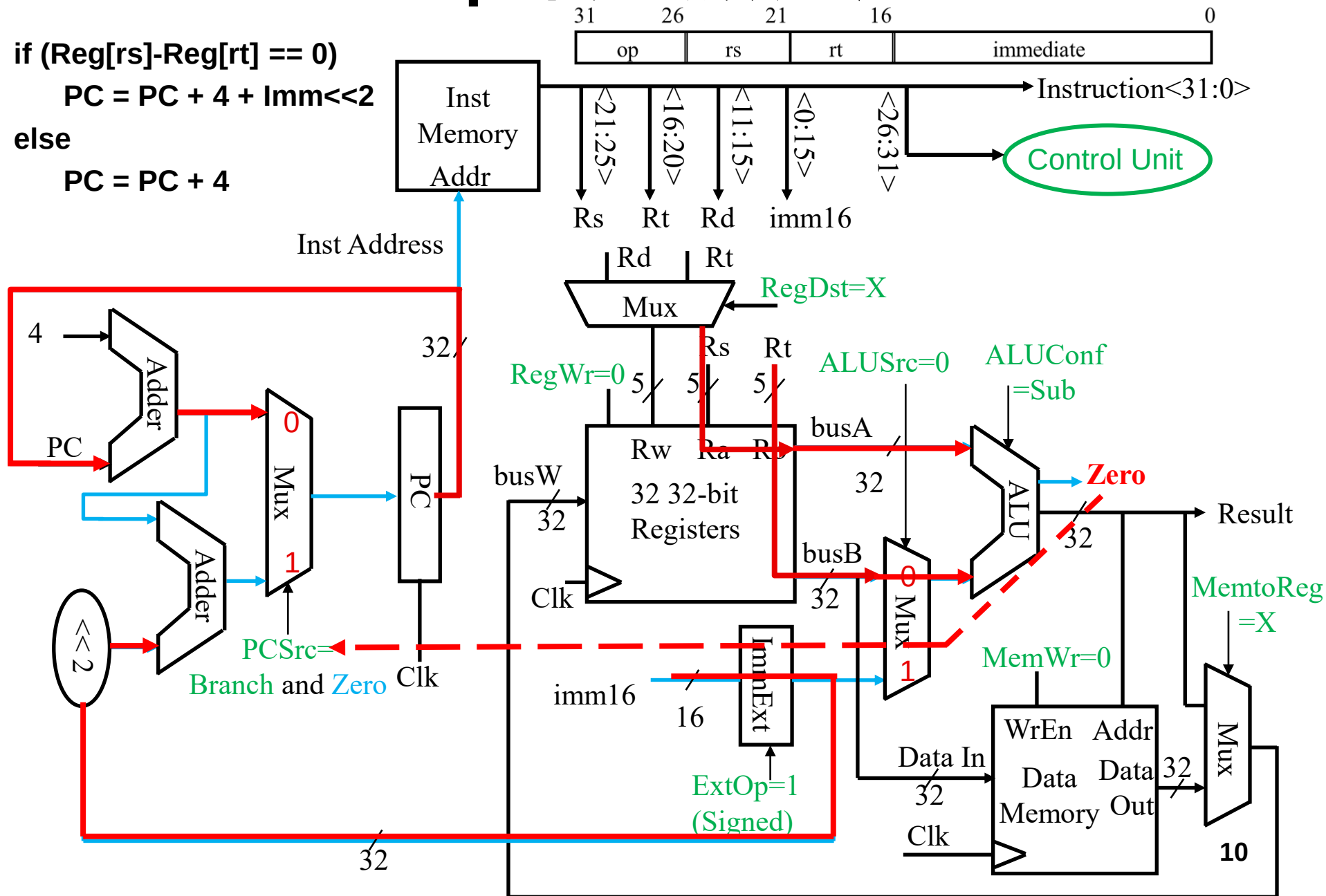
$\text{Reg}[\text{rd}] = \text{Reg}[\text{rs}] + \text{Reg}[\text{rt}]$



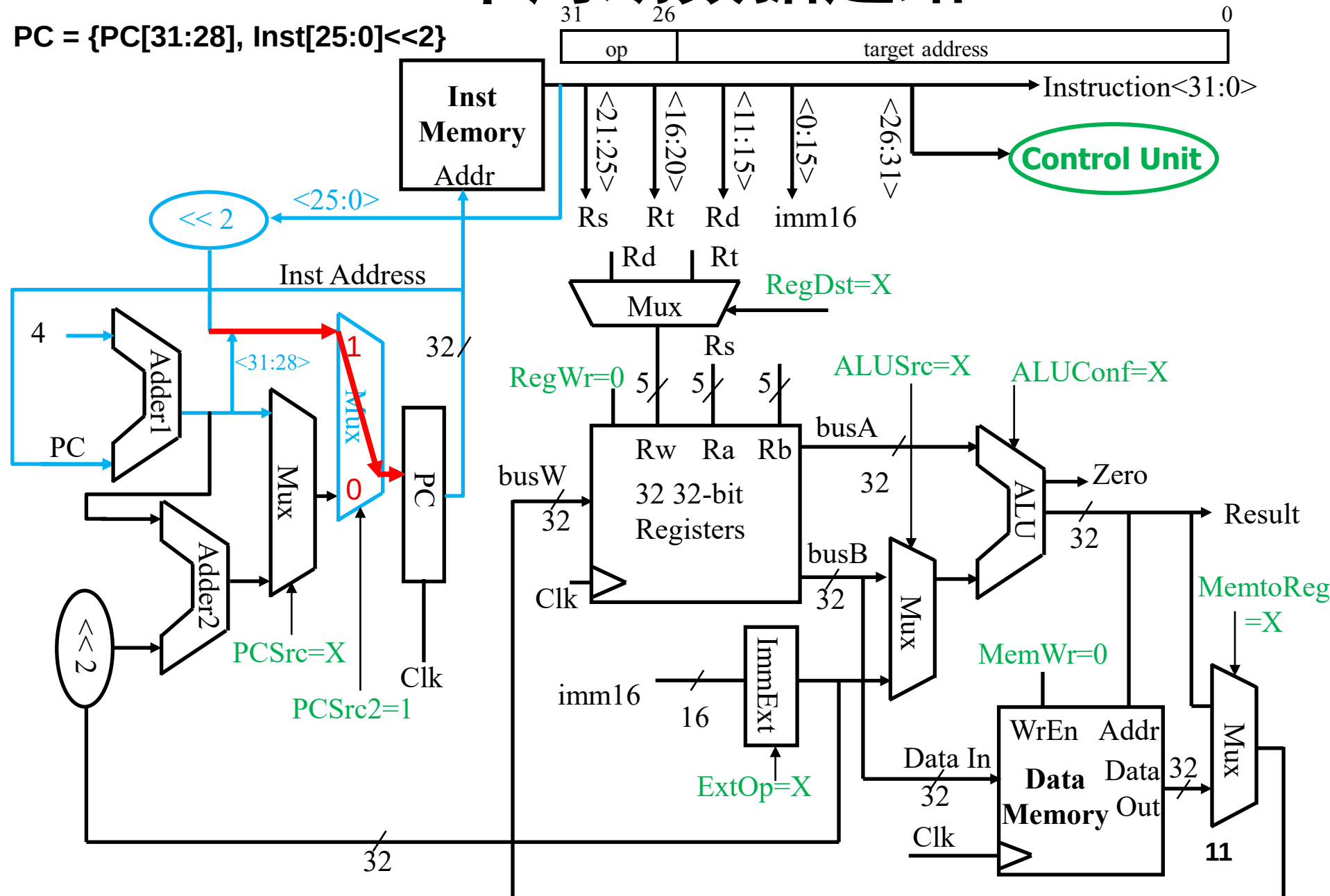
Store: 单周期数据通路



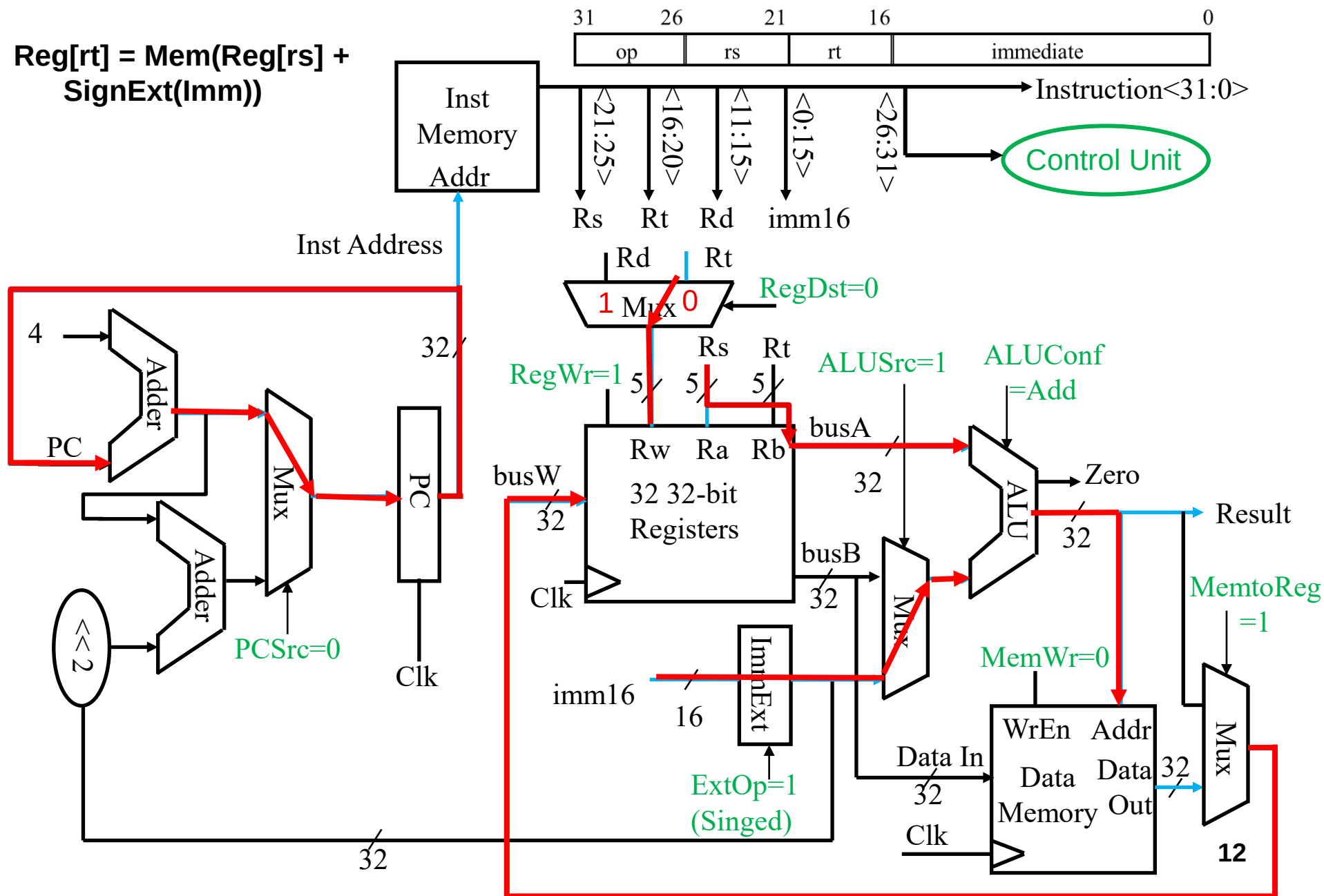
Beq: 单周期数据通路



J: 单周期数据通路

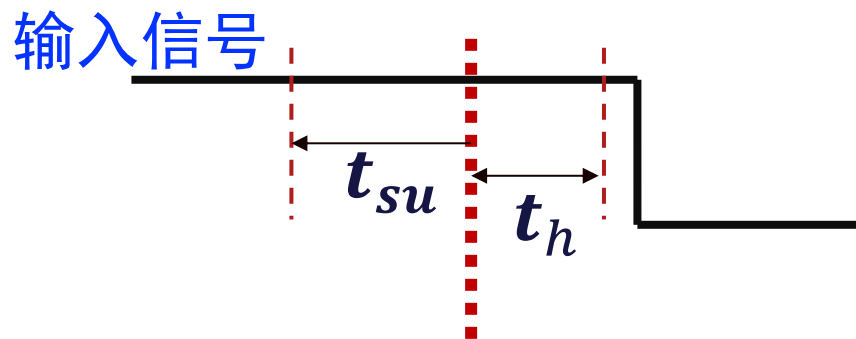


Load: 单周期数据通路



回顾：建立时间和保持时间

- 输出响应时间 t_{co} (co: clk-to-output)
 - 从开始采样到输出信号的延时
- 建立时间 t_{su}
 - 锁存操作开始之前，输入信号应保持不变的最短时间
- 保持时间 t_h
 - 锁存操作之后，输入信号还应保持不变的最短时间



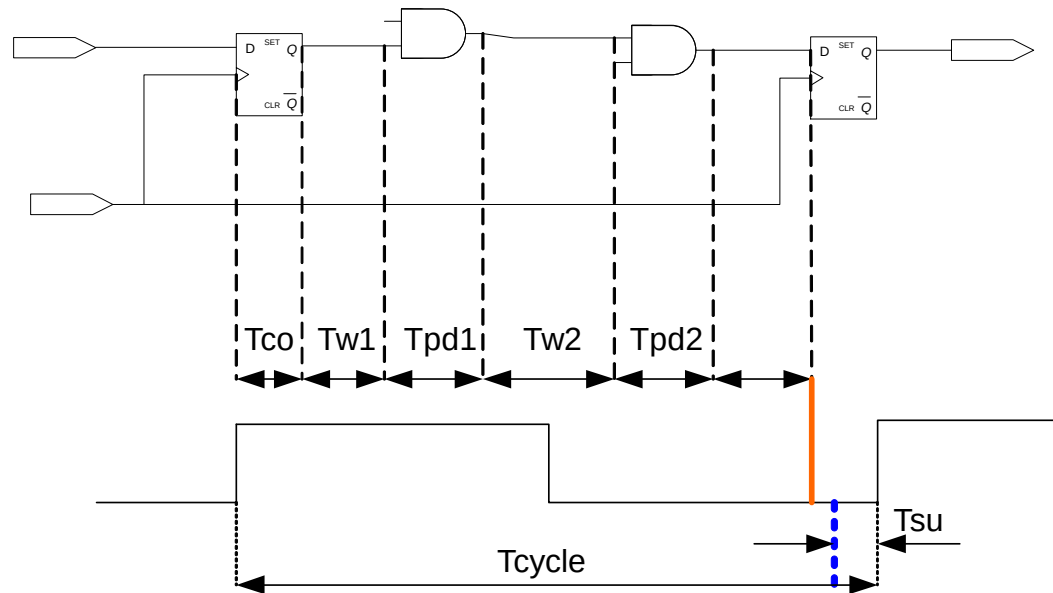
锁存操作发生时刻

回顾：建立时间约束

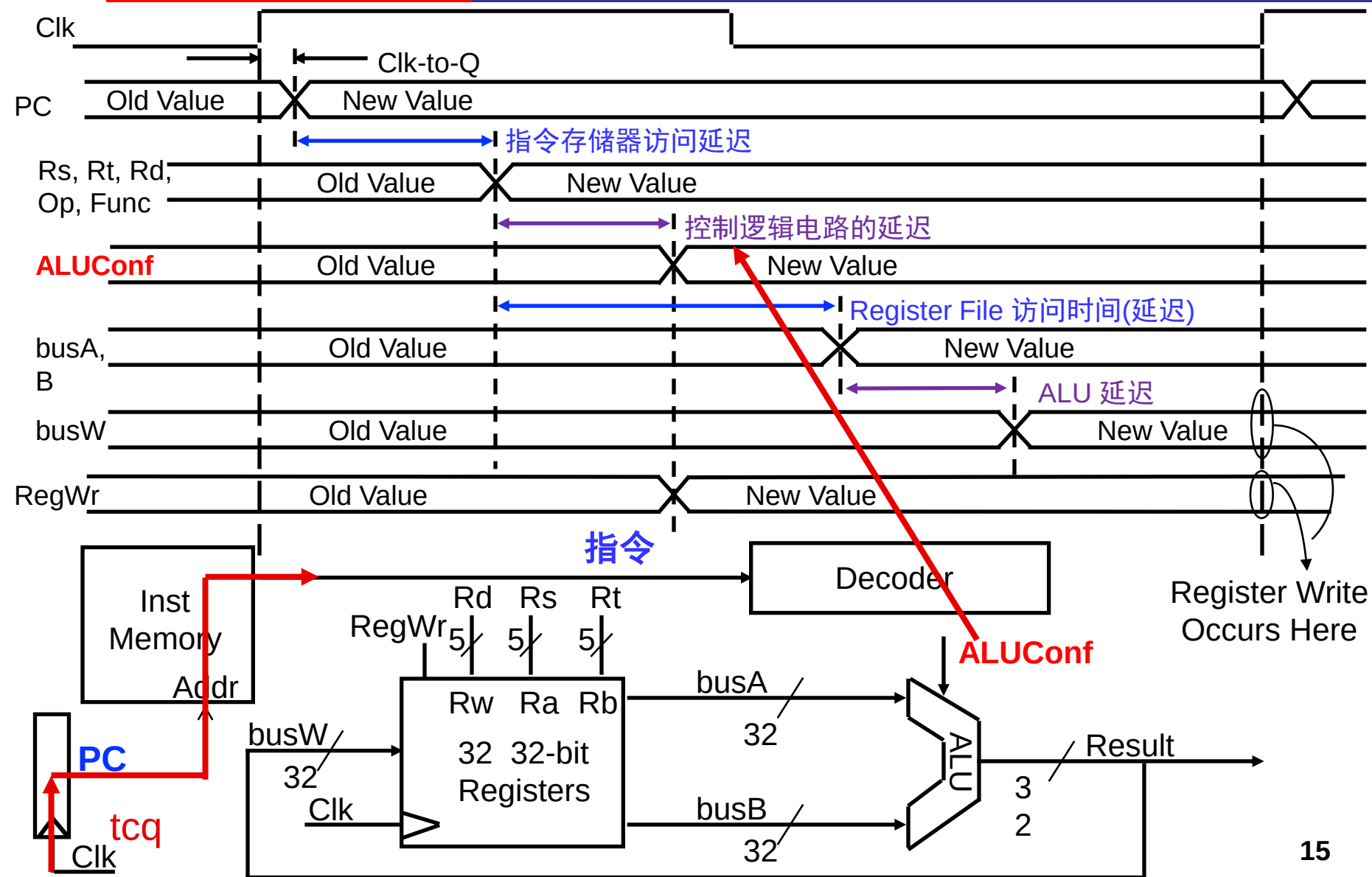
延时需满足： $T_{co} + T_{d_max} + T_{su} \leq T_{cycle}$

(寄存器延时)

(组合逻辑与互连线最长路径延时)



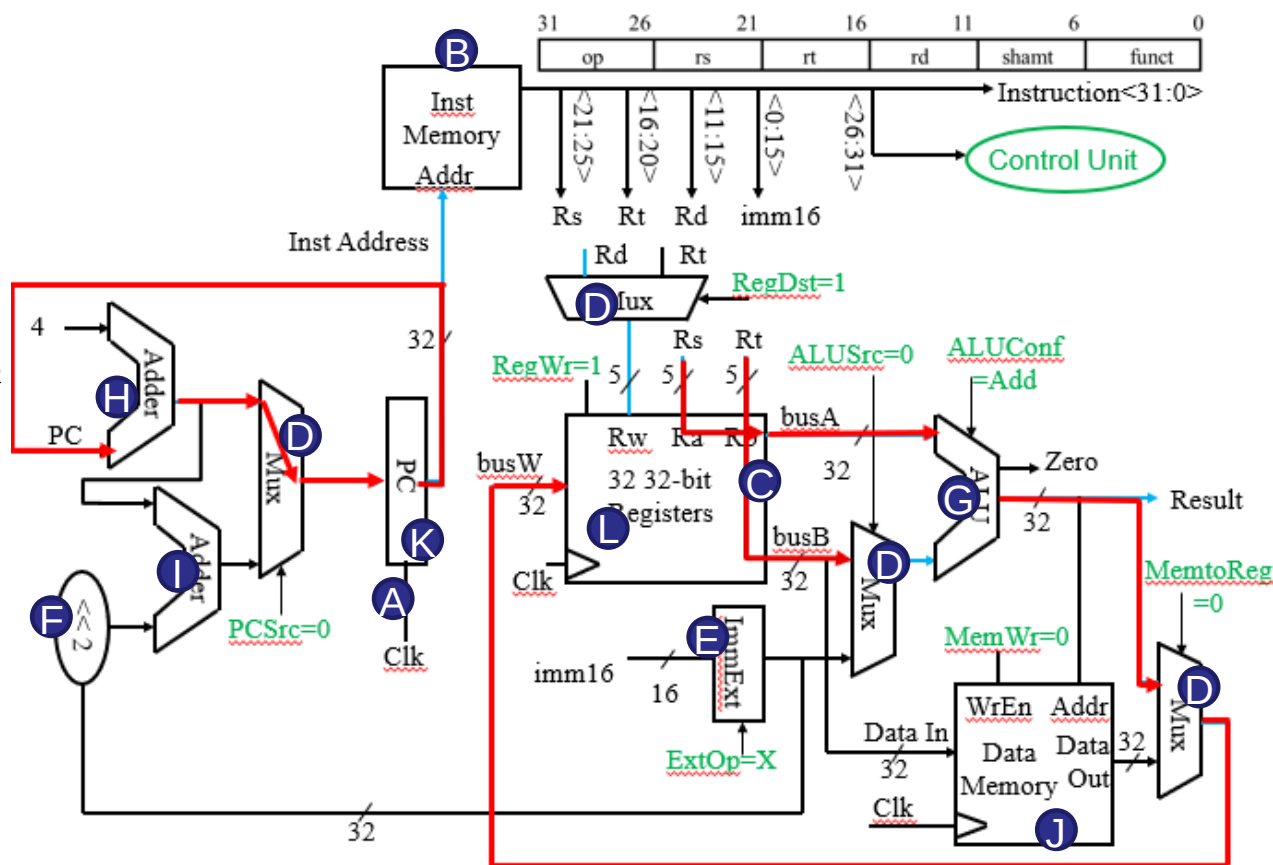
ADD: 如何确定最短时钟周期?



举例：ADD指令的critical path

关键路径：A-B-C-D-G-D-L

- A. PC Clk-to-Q
- B. 指令存储器的访问时间
- C. RF的访问时间
- D. 多路选择器Mux
- E. ImmExt对立即数进行符号位扩展
- F. 移位单元 (<<2)
- G. ALU进行32位运算的时间
- H. PC Adder1计算出PC+4
- I. PC Adder2将PC+4与偏移量相加
- J. 数据存储器访问时间(延迟)
- K. PC建立时间
- L. RF写入的建立时间

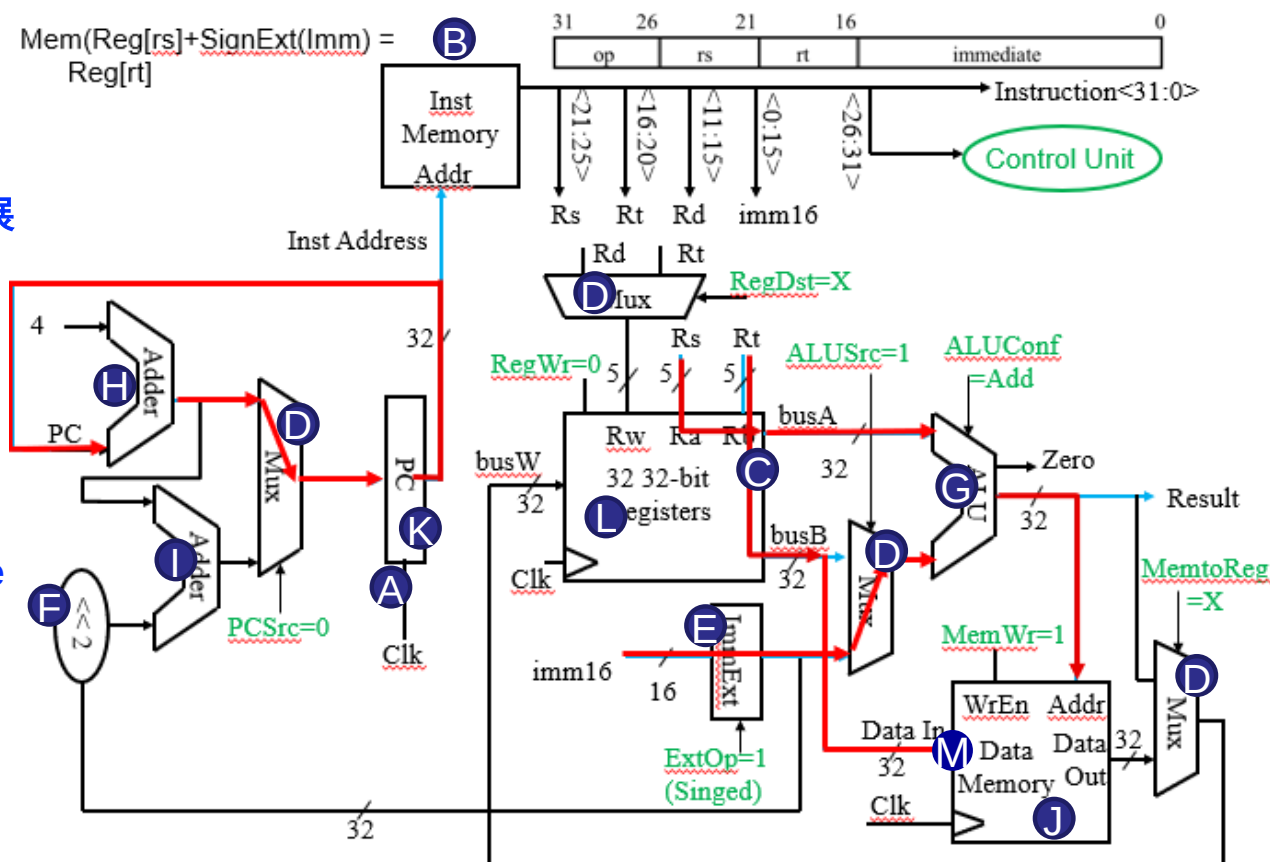


举例：Store指令的critical path

Critical path: A – B – [C or (E – D)] – G – M

取决于C和E+D延时谁更长

- A. PC Clk-to-Q
- B. 指令存储器的访问时间
- C. RF的访问时间
- D. 多路选择器Mux
- E. ImmExt对立即数进行符号位扩展
- F. 移位单元(<<2)
- G. ALU进行32位运算的时间
- H. PC Adder1计算出PC+4
- I. PC Adder2将PC+4与偏移量相加
- J. 数据存储器访问时间(延迟)
- K. PC建立时间
- L. RF写入的建立时间
- M. Data memory Addr setup time

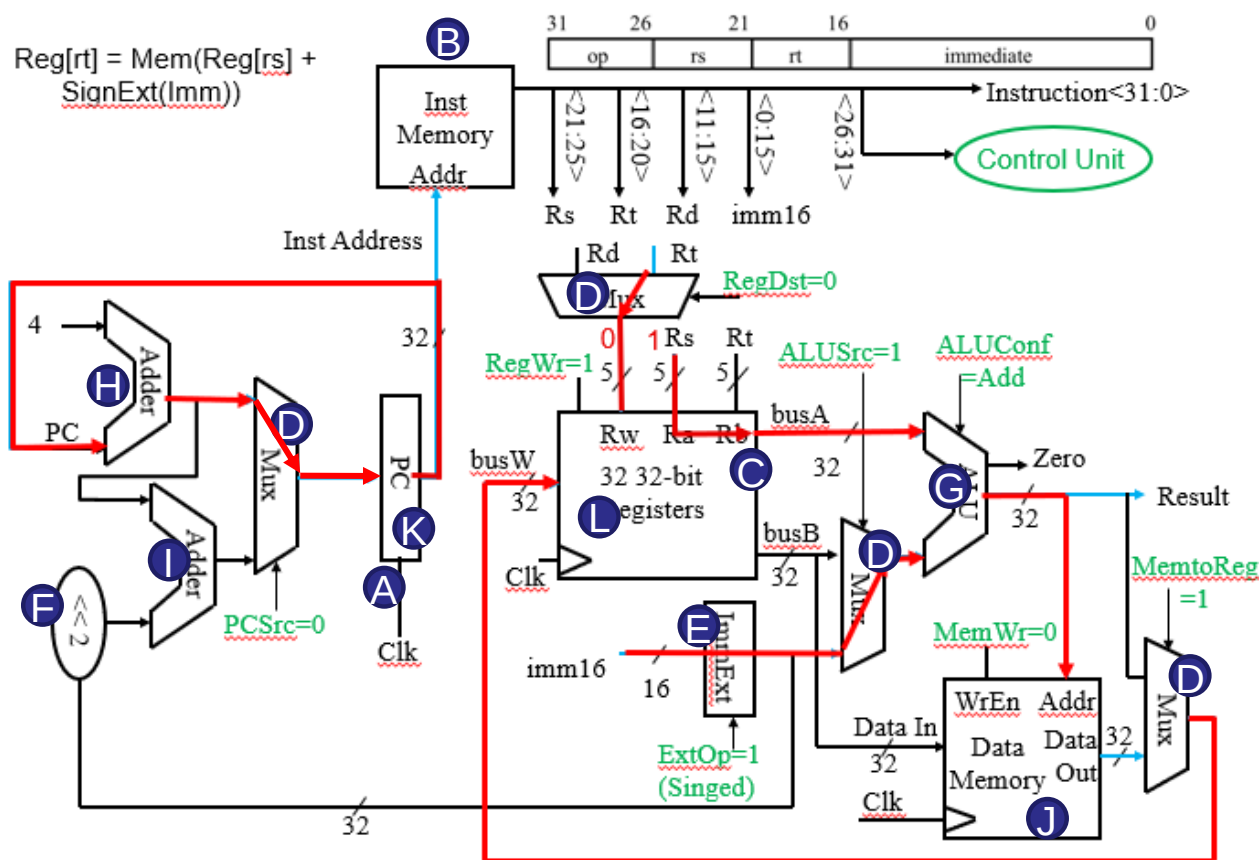


举例：Load指令的critical path

关键路径：A – B – [C or (E – D)] – G – J – D – L

取决于C和E+D延时谁更长

- A. PC Clk-to-Q
- B. 指令存储器的访问时间
- C. RF的访问时间
- D. 多路选择器Mux
- E. ImmExt对立即数进行符号位扩展
- F. 移位单元 (<<2)
- G. ALU进行32位运算的时间
- H. PC Adder1计算出PC+4
- I. PC Adder2将PC+4与偏移量相加
- J. 数据存储访问时间(延迟)
- K. PC建立时间
- L. RF写入的建立时间

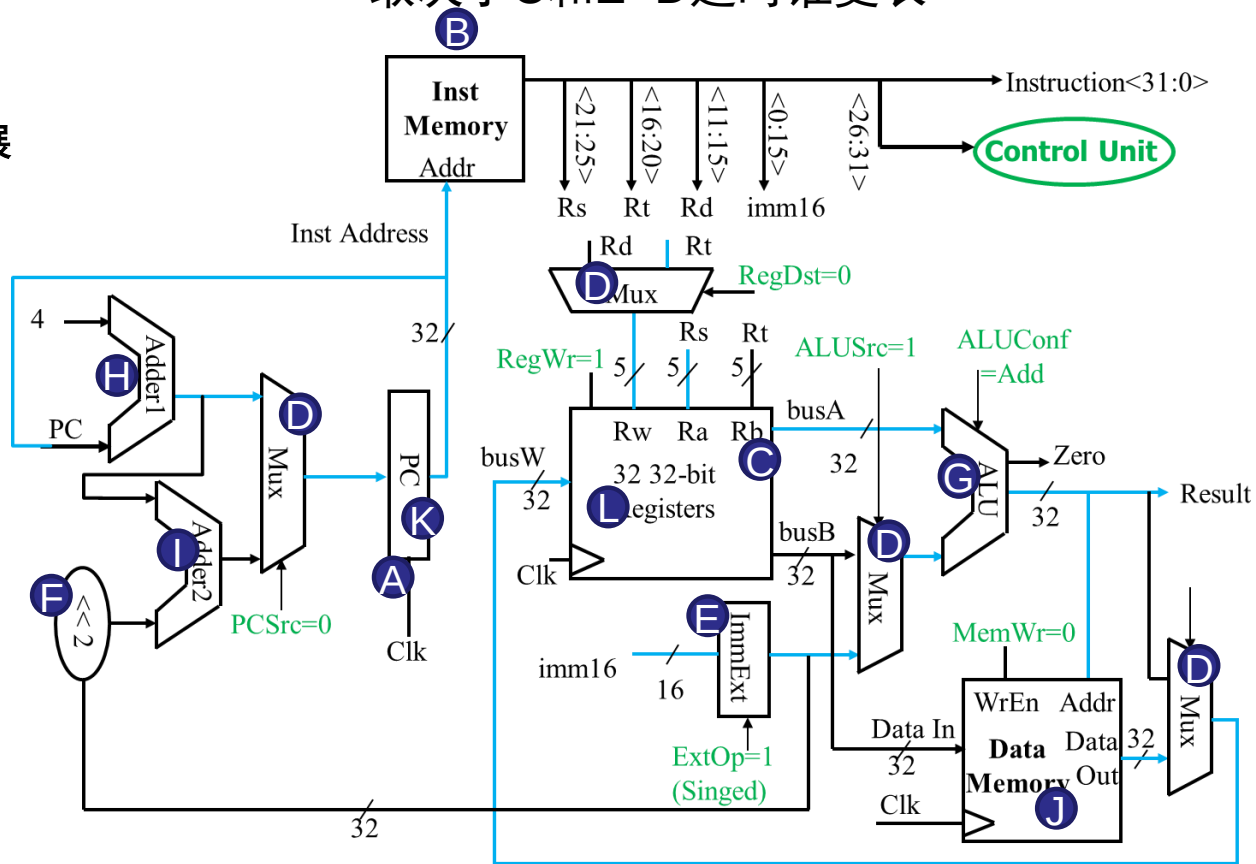


最长的critical path

关键路径: A – B – [C or (E – D)] – G – J – D – L

- A. PC Clk-to-Q
- B. 指令存储器的访问时间
- C. RF的访问时间
- D. 多路选择器Mux
- E. ImmExt对立即数进行符号位扩展
- F. 移位单元 (<<2)
- G. ALU进行32位运算的时间
- H. PC Adder1计算出PC+4
- I. PC Adder2将PC+4与偏移量相加
- J. 数据存储器访问时间(延迟)
- K. PC建立时间
- L. RF写入的建立时间

取决于C和E+D延时谁更长



最长critical path?

1. ADDU
2. BEQ
3. LW
4. ORI
5. SUBU
6. SW

最长的critical path: LW

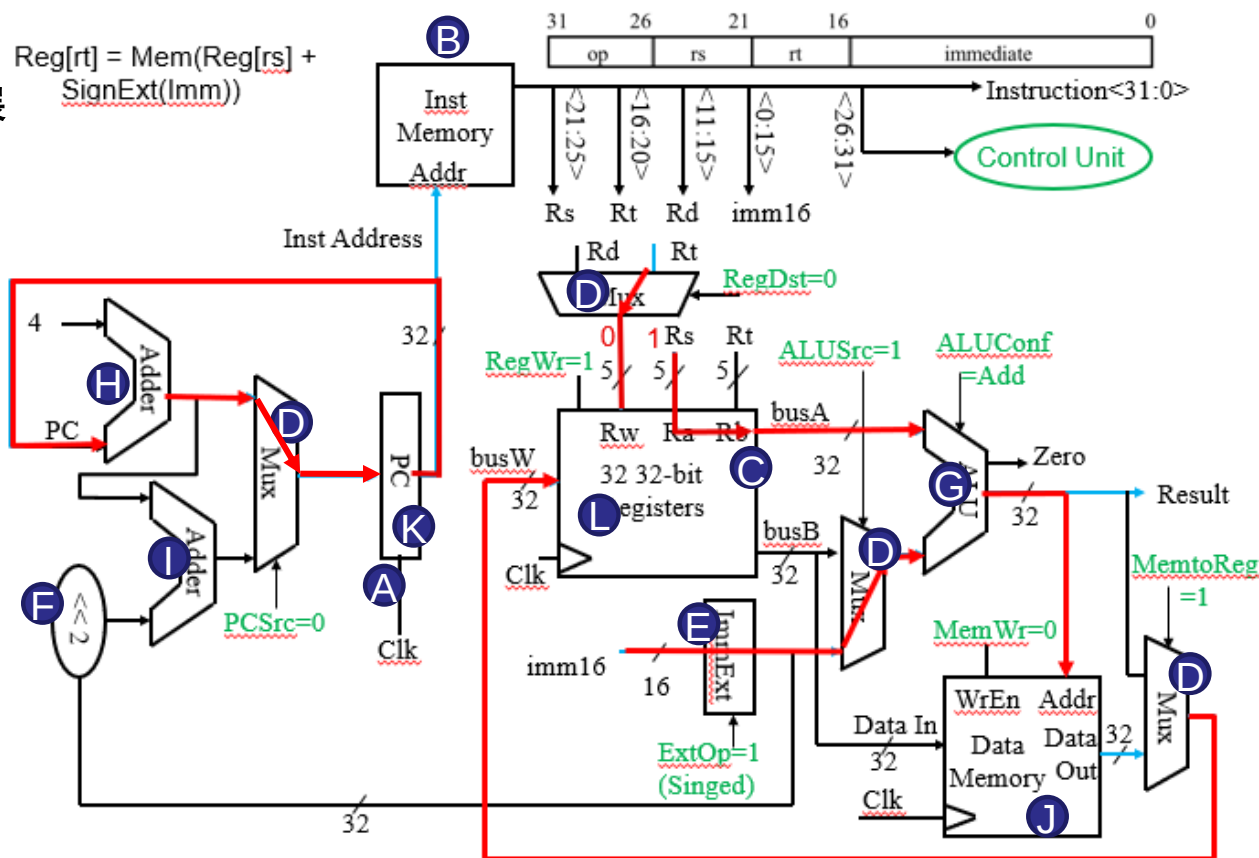
关键路径: A – B – [C or (E – D)] – G – J – D – L

取决于C和E+D延时谁更长

- A. PC Clk-to-Q
- B. 指令存储器的访问时间
- C. RF的访问时间
- D. 多路选择器Mux
- E. ImmExt对立即数进行符号位扩展
- F. 移位单元 (<<2)
- G. ALU进行32位运算的时间
- H. PC Adder1计算出PC+4
- I. PC Adder2将PC+4与偏移量相加
- J. 数据存储访问时间(延迟)
- K. PC建立时间
- L. RF写入的建立时间

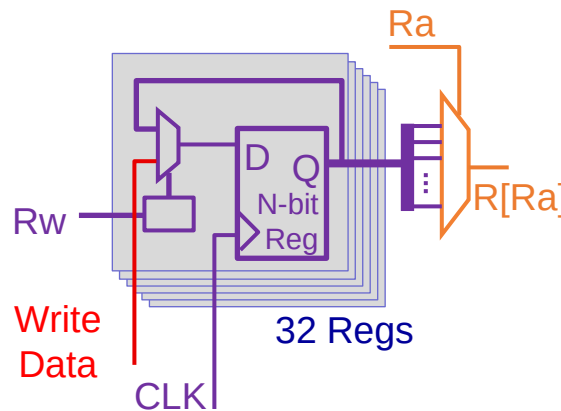
最长critical path?

1. ADDU
2. BEQ
3. LW
4. ORI
5. SUBU
6. SW



寄存器堆访问

- 时钟触发边沿写入，且支持“先写后读/写后即读”
 - 时钟沿写入到RF的数据在下一个边沿之前就可以直接被访问
 - RF/Memory阵列大，写入的更新数据需建立时间，如50/200ps
 - 如刚写入的数据马上就要读出使用，可利用额外旁路加速（见附录）



单周期实现的性能

- 假设主要功能单元的操作时间
 - ALU和加法器：100ps
 - 存储单元（读写相同）：200ps
 - 寄存器堆（读写相同）：50ps
 - 假设其他延迟忽略不计
- 假设某程序中，各种指令的占比
 - Lw: 25%
 - Sw: 10%
 - ALU: 45%
 - Beq: 15%
 - J: 5%
- 该程序平均指令执行时间是多少？
 - 需要计算出每一类指令所需时间

回顾指令的单周期执行过程

- 不同的指令执行需要不同的步骤(记住简写)
 - 取指令 (Instruction Fetch, **IF**)
 - 指令译码和控制信号生成 (Instruction Decode, **ID**)
 - 操作执行 (Execution, **EX**)
 - 内存访问 (Memory Access, **MEM**)
 - 写回寄存器堆 (Write Back, **WB**)
- 不同的执行步骤有不同的累积延时
 - 关键路径 (Critical Path)

单周期实现的性能

- 不同指令的关键路径

指令类型	指令用到的功能单元				
R型	取指令	指令译码和寄存器堆读取	ALU运算	寄存器堆写入	
Load类型	取指令	指令译码和寄存器堆读取	ALU运算	存储器读取	寄存器堆写入
Store类型	取指令	指令译码和寄存器堆读取	ALU运算	存储器写入	
Branch	取指令	指令译码和寄存器堆读取	ALU运算		
Jump	取指令	指令译码和寄存器堆读取			

单周期实现的性能

- 各类的指令的执行时间细分
 - 如果所有指令都在**相同的时钟周期内**完成，则时钟周期取决于**执行时间最长**的指令：600ps

指令类型	指令存储器	读寄存器堆	ALU运算	数据存储器	写寄存器堆	总延时
R型	200ps	50ps	100ps	0	50ps	400ps
Load类型	200ps	50ps	100ps	200ps	50ps	600ps
Store类型	200ps	50ps	100ps	200ps	0	550ps
Branch	200ps	50ps	100ps	0	0	350ps
Jump	200ps	0	0	0	0	200ps

从单周期到多周期

- 给定任务，衡量处理器性能的方式
 - $\text{CPU执行时间} = \text{指令数} \times \text{CPI} \times \text{时钟周期}$
- 减少CPU执行时间的主要途径
 - 减小指令数
 - 减小CPI 时钟周期
- 改进方法：多周期处理器
 - 关键路径延时被拆分为N个时钟，时钟周期减少为 $1/N$
 - 每类指令用不同的时钟数完成
 - 不包含全部步骤的指令执行的周期数小于N
 - 如：LW五周期，J两周期
 - 平均CPI小于N， $\text{CPI} \times \text{时钟周期}$ 减小

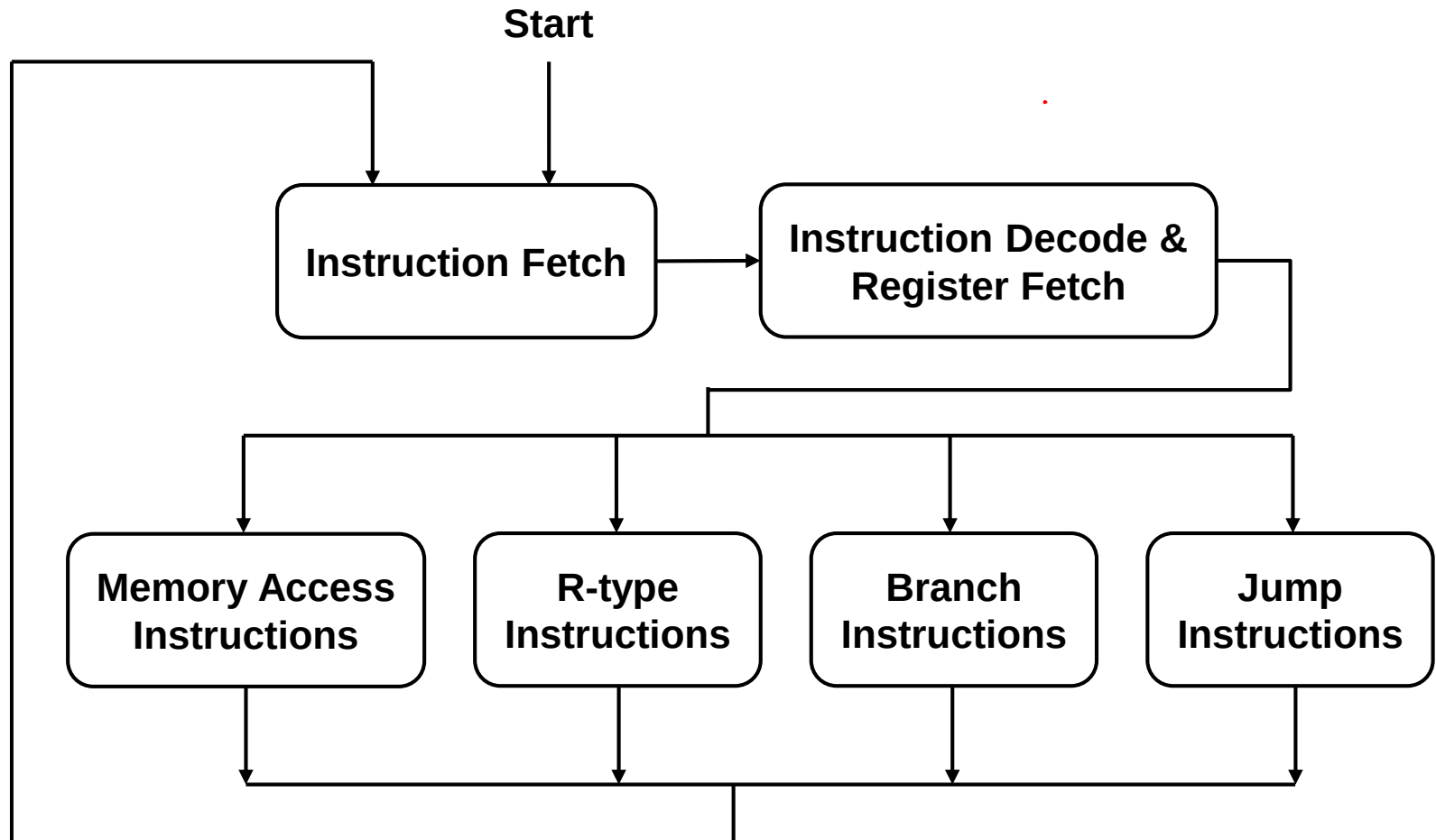
多周期：基本思路

- 多周期 (Multicycle)
 - 一条指令的执行分解为多个步骤
 - ✓ 取指令
 - ✓ 指令译码，以及可能的读寄存器
 - ✓ 其他步骤（不同指令可能不同）
 - 指令的每一步占用一个时钟周期
 - 同一个功能单元可以在多个步骤中复用

本节课目录

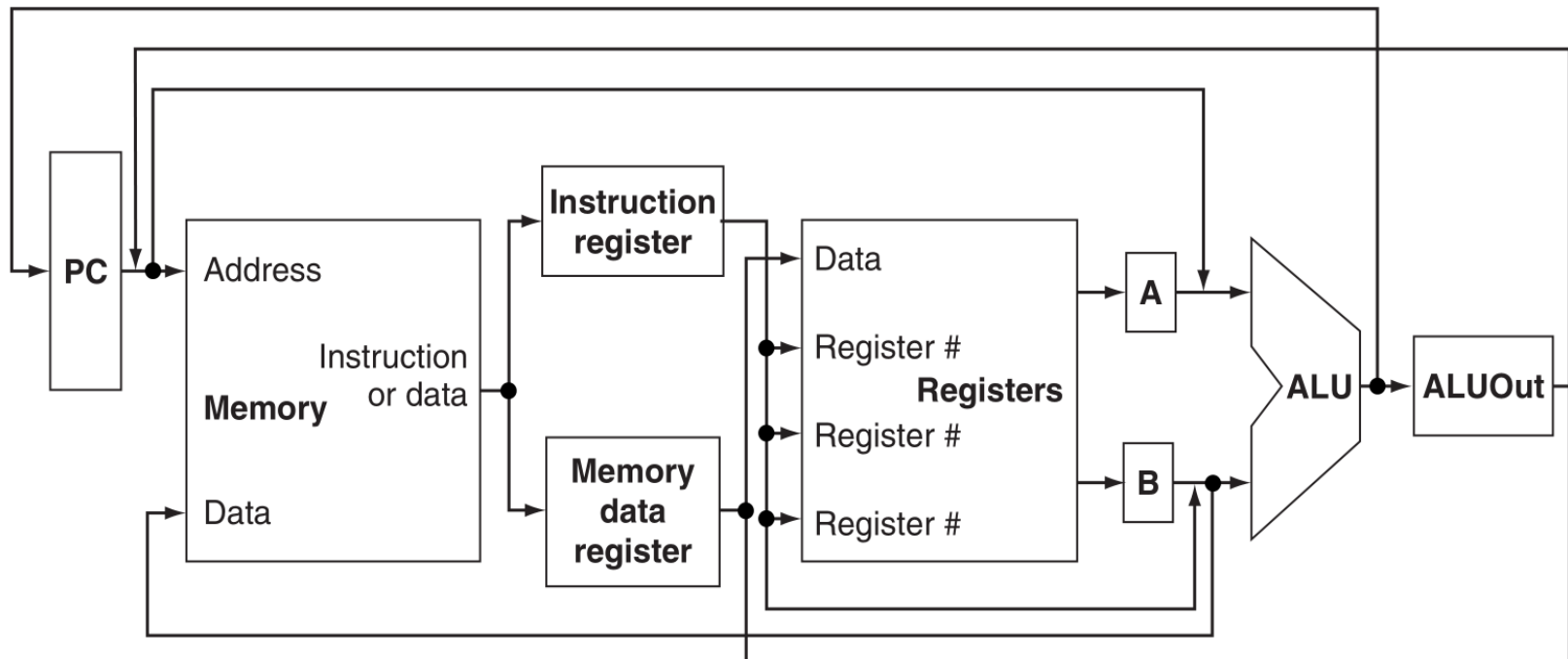
- 复习与回顾
- 单周期数据通路设计流程
 - 指令集和设计需求分析
 - 功能模块设计
 - 数据通路模块组装
 - 控制信号分析与逻辑设计
- 单周期数据通路延时分析
- 多周期数据通路
- 异常与中断

多周期功能分解



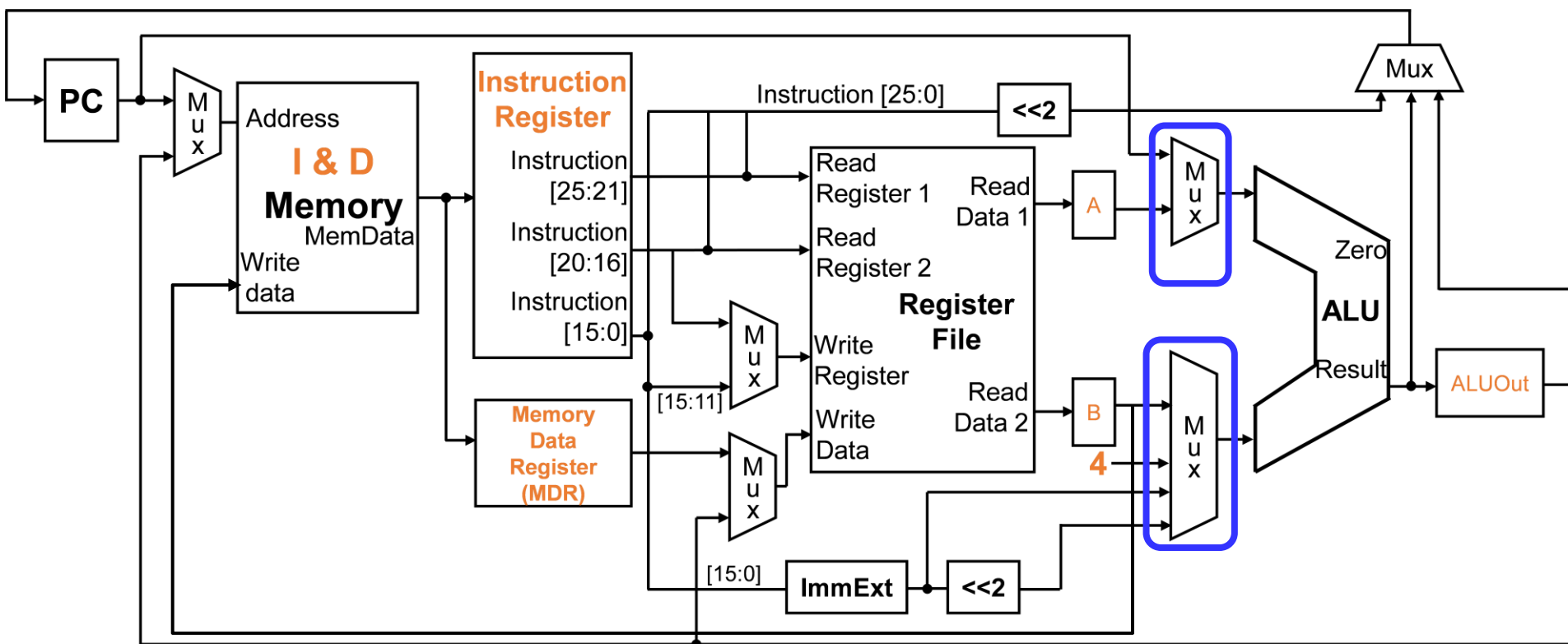
多周期处理器数据通路

- 与单周期的主要结构差别
 - 阶段性存储：主要功能单元加输出寄存器存储输出值
 - 结构复用：
 - 指令和数据共用一个存储单元
 - 只需要一个ALU，而不是一个ALU和两个加法器

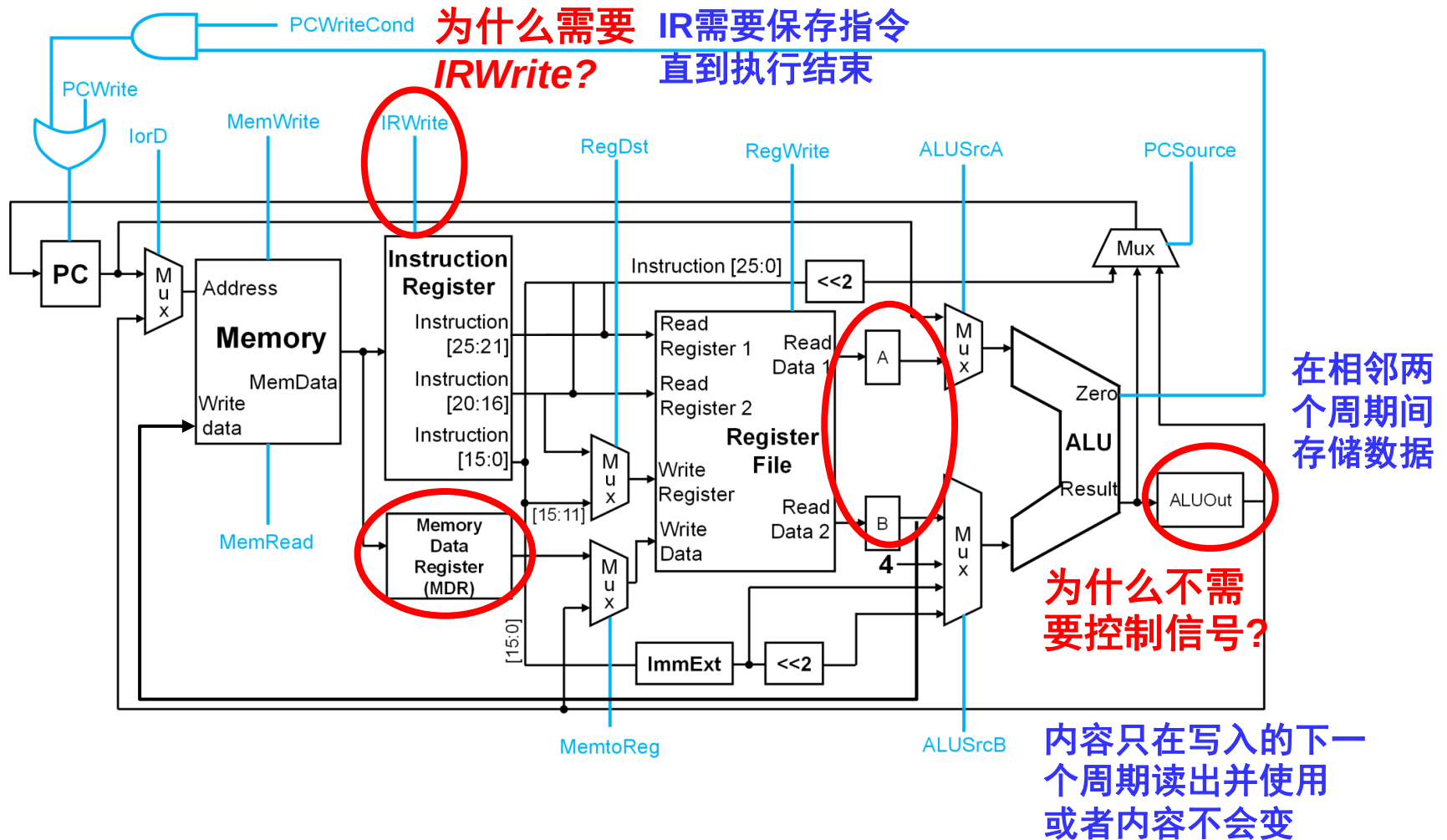


多周期数据通路

- 给ALU的第一个输入添加一个多路复用器，在寄存器A和PC间进行选择
- ALU第二个输入的多路复用器由2路改为4路，两个新加输入为：常数4（用于PC增值）和符号扩展及移位后的偏移量字段（用于计算分支地址）



多周期数据通路



多周期功能描述

• 有限状态机的视角 – 五个阶段

0: Instruction Fetch

MemRead
IRWrite
PCWrite
PCSource=00
ALUSrcA=0
IorD=0
ALUSrcB=01
ALUOp=00

步骤\指令类型	R型指令	内存访问指令	分支指令	跳转指令
取指令	$IR \leq \text{Memory}[PC]$ $PC \leq PC+4$ (提前完成)			
译码 & 寄存器 堆读取	$A \leq \text{Reg}[IR[25:21]]$; $B \leq \text{Reg}[IR[20:16]]$ $ALUOut \leq PC + (\text{sign-extended}(IR[15:0]) \ll 2)$ (提前完成)			
执行 & 分支/跳转完成	$ALUOut \leq A \text{ op } B$	$ALUOut \leq A + \text{sign-extend}(IR[15:0])$	if (A==B) $PC \leq ALUOut$	$PC \leq \{PC[31:28], IR[25:0], 2'b00\}$
内存访问 & R型指令完成	$\text{Reg}[IR[15:11]] \leq ALUOut$	Load: $MDR \leq \text{Memory}[ALUOut]$ Store: $\text{Memory}[ALUOut] \leq B$		
内存		Load: $\text{Reg}[IR[20:16]] \leq MDR$		

```
MemRead
IRWrite
PCWrite
PCSource=00
ALUSrcA=0
IorD=0
ALUSrcB=01
ALUOp=00
```

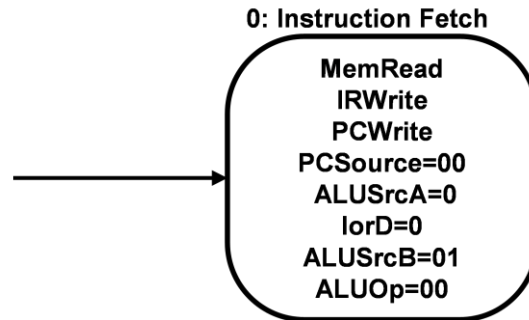
PCSource=00



多周期的有限状态机视角

- **State 0: Instruction Fetch**

- IRWrite
- PCWrite
- PCSource=00
- ALUSrcA=0
- IorD=0
- ALUSrcB=01
- ALUOp=00
- MemRead
- 思考：MemRead=0时，Memory模块的输出是什么？
 - Memory模块输出保持不变？高阻输出？
 - 由设计人员考虑，实践中均可，但要注意不同实现方式的影响



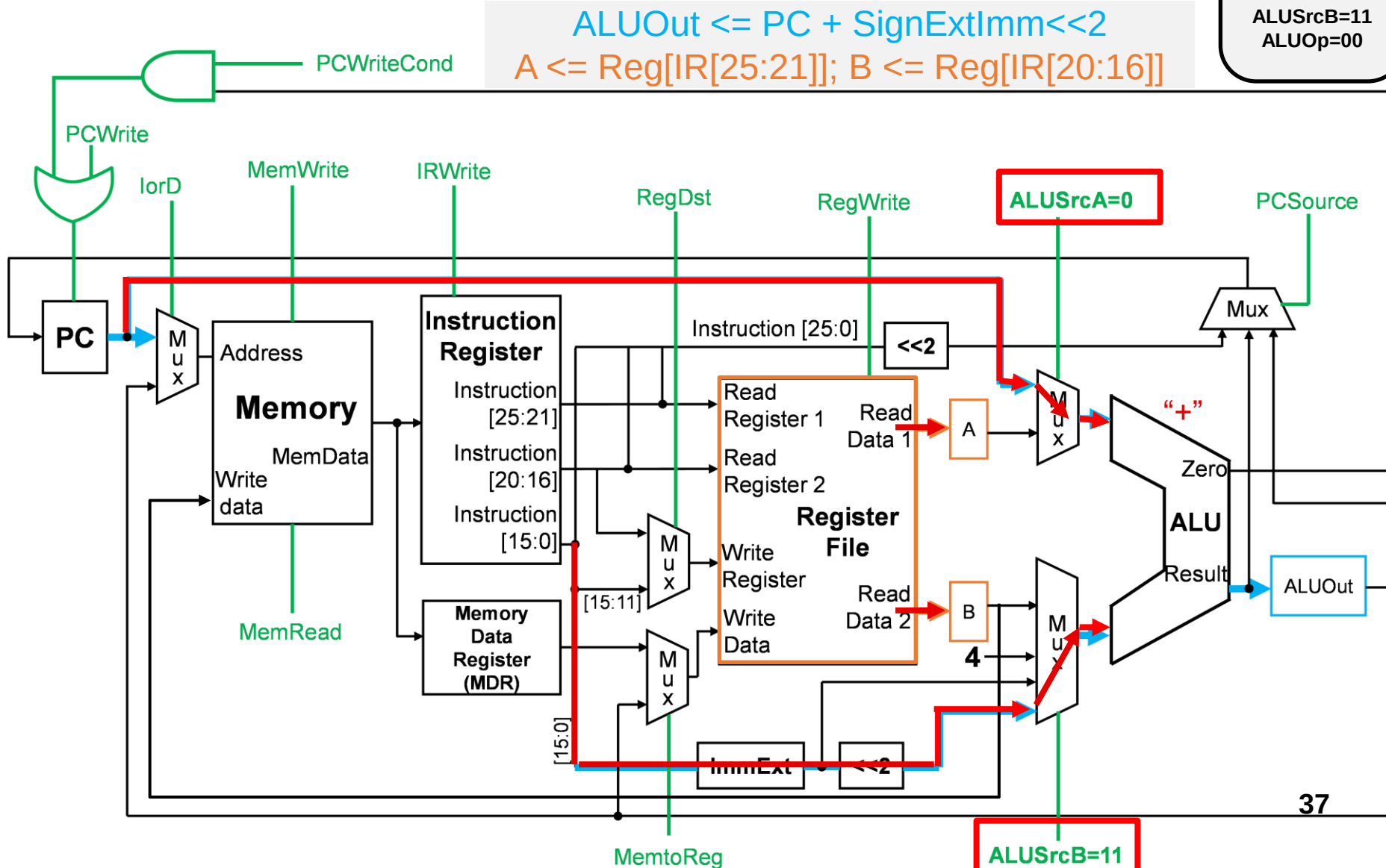
多周期的有限状态机视角

- **State 1: Instruction decode/Register fetch**
 - ALUSrcA=0
 - ALUSrcB=11
 - ALUOp=00 (add)

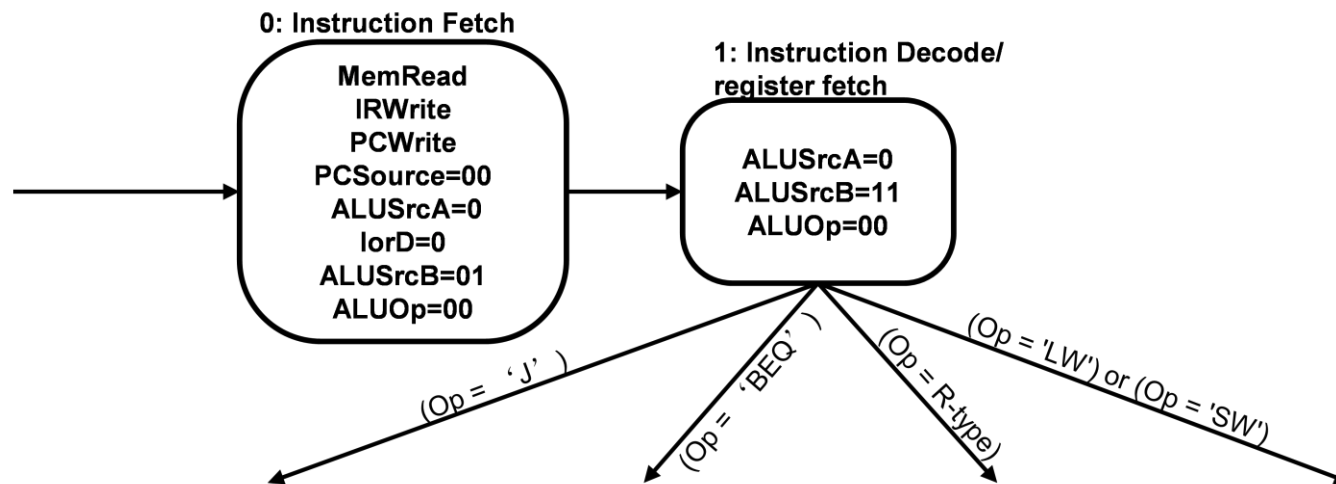
步骤\指令类型	R型指令	内存访问指令	分支指令	跳转指令
取指令	IR <= Memory[PC] PC <= PC+4			
译码 & 寄存器堆读取	A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]] ALUOut <= PC + (sign-extended(IR[15:0]) << 2)			
执行 & 分支/跳转完成	ALUOut <= A op B	ALUOut <= A + sign-extend(IR[15:0])	if (A==B) PC<=ALUOut	PC <= {PC[31:28], IR[25:0], 2'b00}
内存访问 & R型指令完成	Reg[IR[15:11]] <= ALUOut	Load: MDR<=Memory[ALUOut] Store: Memory[ALUOut] <= B		
内存		Load: Reg[IR[20:16]]<=MDR		36

多周期功能描述：寄存器读取

ALUSrcA=0
ALUSrcB=11
ALUOp=00



From State 1 to more states...



Branch型指令的状态转移

- 有限状态机的视角
 - 还有1个状态

8: Branch completion

PCWriteCond
ALUSrcA=1
ALUSrcB=00
ALUOp=01
PCSource=01

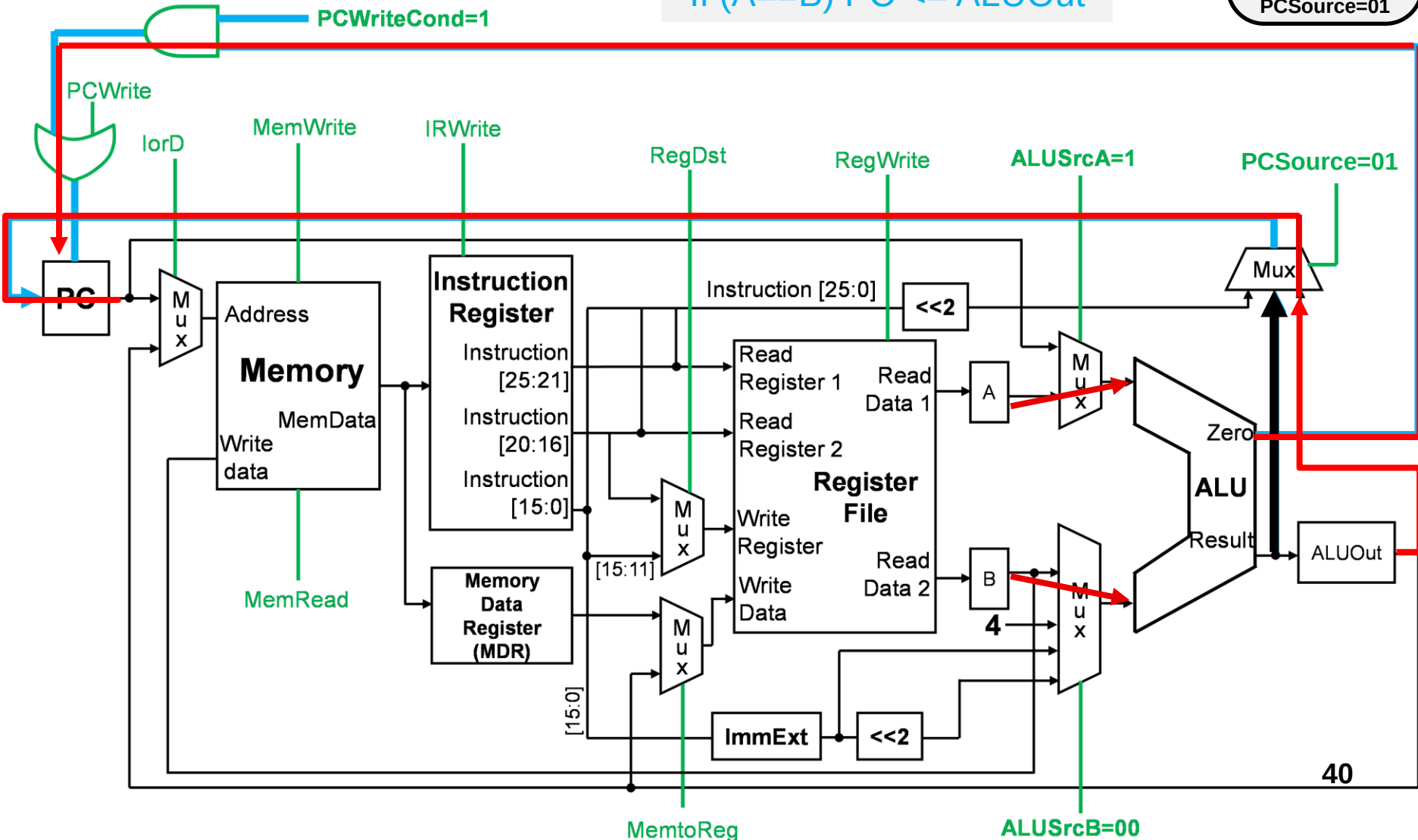


步骤\指令类型	R型指令	内存访问指令	分支指令	跳转指令
取指令	$IR \leq \text{Memory}[PC]$ $PC \leq PC+4$			
译码 & 寄存器堆读取	$A \leq \text{Reg}[IR[25:21]]$; $B \leq \text{Reg}[IR[20:16]]$ $ALUOut \leq PC + (\text{sign-extended}(IR[15:0]) \ll 2)$			
执行 & 分支/跳转完成	$ALUOut \leq A \text{ op } B$	$ALUOut \leq A + \text{sign-extend}(IR[15:0])$	if (A==B) $PC \leq ALUOut$	$PC \leq \{PC[31:28], IR[25:0], 2'b00\}$
内存访问 & R型指令完成	$\text{Reg}[IR[15:11]] \leq ALUOut$	Load: $MDR \leq \text{Memory}[ALUOut]$ Store: $\text{Memory}[ALUOut] \leq B$		
内存		Load: $\text{Reg}[IR[20:16]] \leq MDR$		

多周期功能描述：Branch

PCWriteCond
ALUSrcA=1
ALUSrcB=00
ALUOp=01
PCSource=01

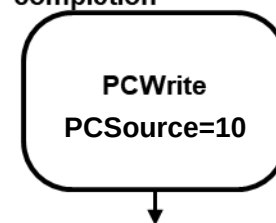
If (A==B) PC <= ALUOut



J型指令的状态转移

- 有限状态机的视角
 - 还有1个状态

9: Jump completion

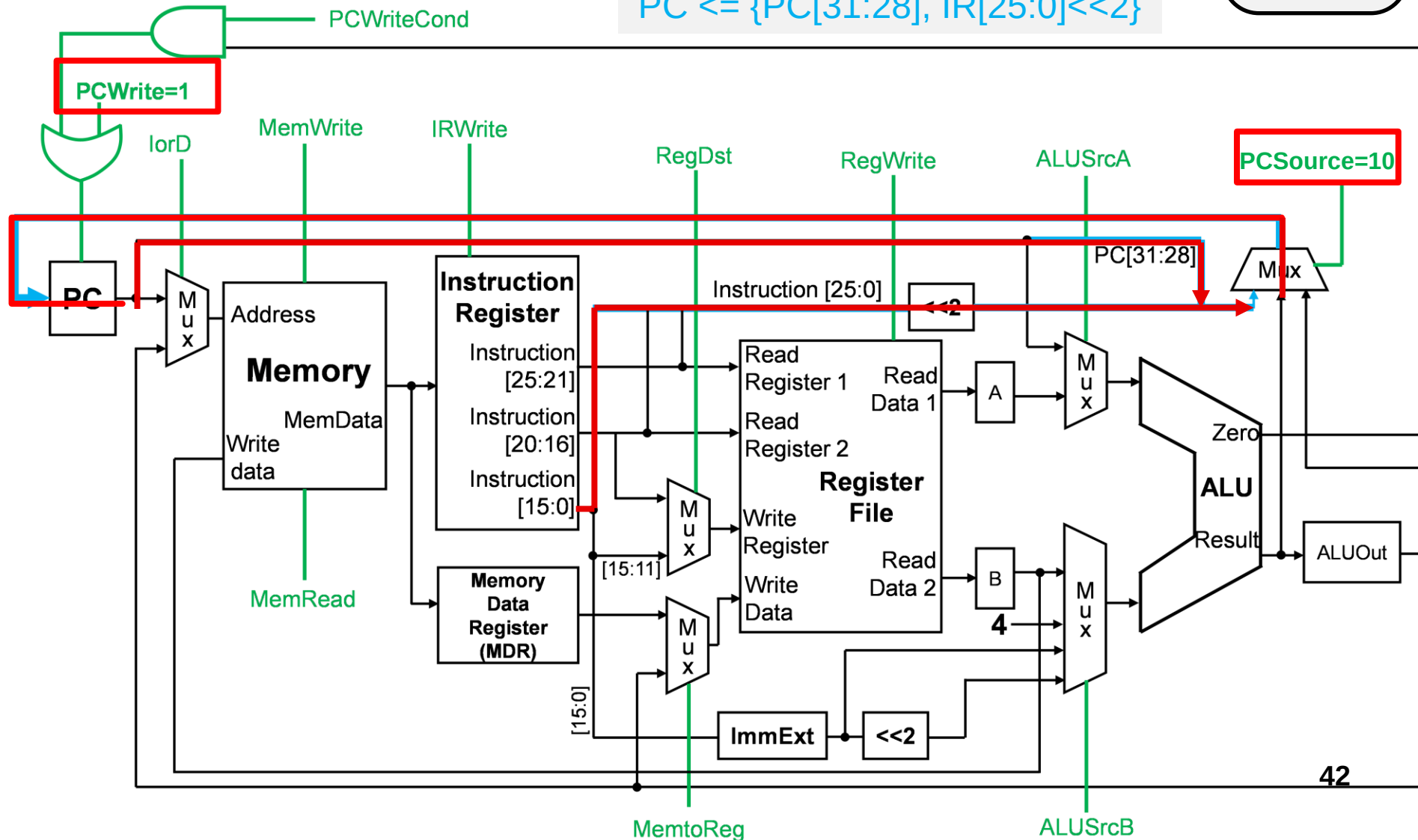


步骤\指令类型	R型指令	内存访问指令	分支指令	跳转指令
取指令	$IR \leq \text{Memory}[PC]$ $PC \leq PC+4$			
译码 & 寄存器堆读取	$A \leq \text{Reg}[IR[25:21]]$; $B \leq \text{Reg}[IR[20:16]]$ $ALUOut \leq PC + (\text{sign-extended}(IR[15:0]) \ll 2)$			
执行 & 分支/跳转完成	$ALUOut \leq A \text{ op } B$	$ALUOut \leq A + \text{sign-extend}(IR[15:0])$	if (A==B) $PC \leq ALUOut$	$PC \leq$ $\{PC[31:28],$ $IR[25:0],$ $2'b00\}$
内存访问 & R型指令完成	$\text{Reg}[IR[15:11]] \leq ALUOut$	Load: $MDR \leq \text{Memory}[ALUOut]$ Store: $\text{Memory}[ALUOut] \leq B$		
内存		Load: $\text{Reg}[IR[20:16]] \leq MDR$		

多周期功能描述：J型指令

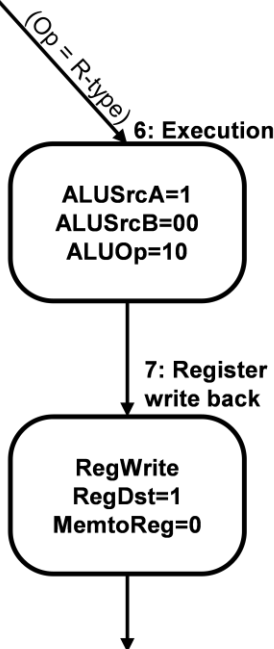
PCWrite
PCSource=10

$PC \leq \{PC[31:28], IR[25:0] \ll 2\}$



R型指令的状态转移

- 有限状态机的视角
 - 还有2个状态

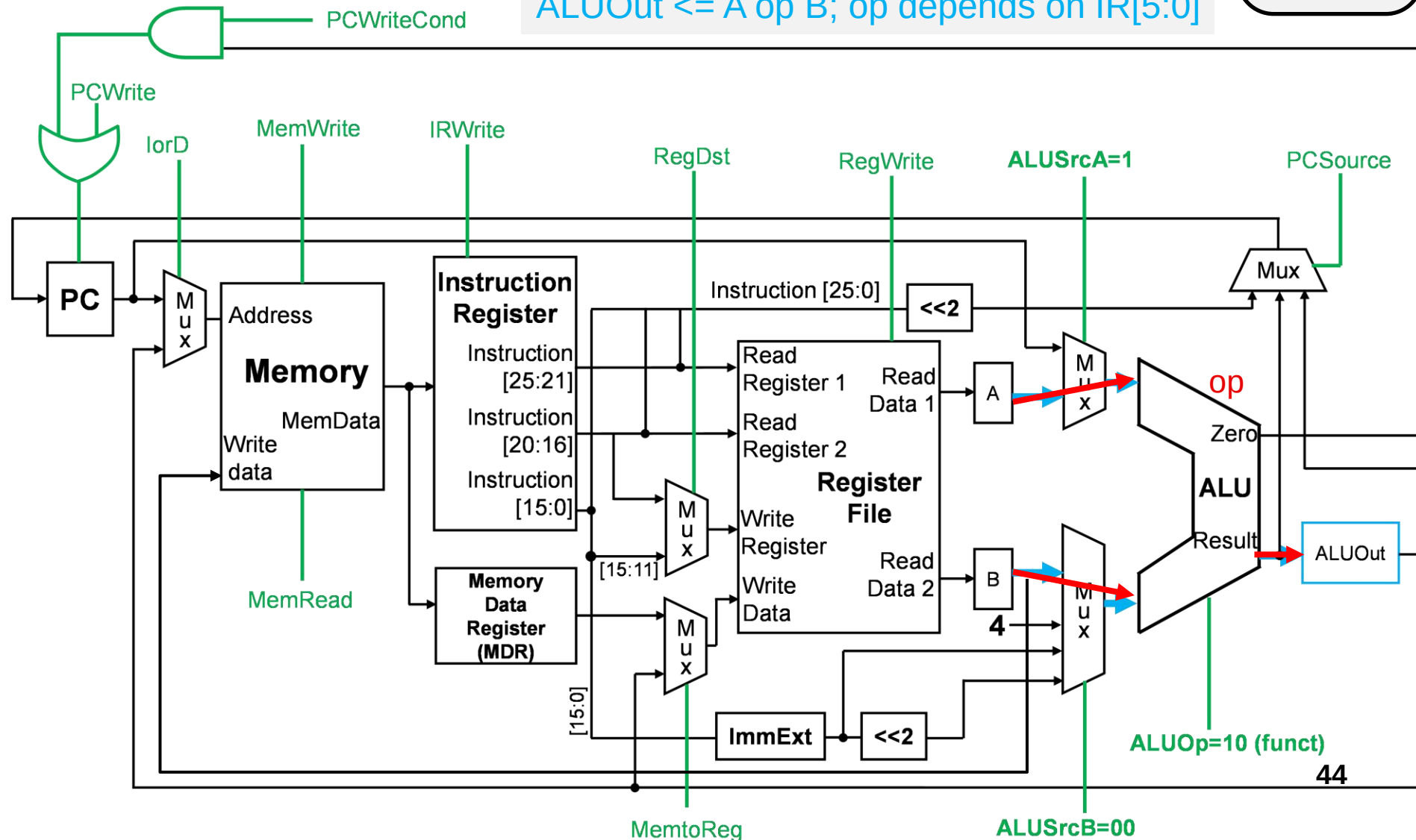


步骤\指令类型	R型指令	内存访问指令	分支指令	跳转指令
取指令	$IR \leq Memory[PC]$ $PC \leq PC+4$			
译码 & 寄存器堆读取	$A \leq Reg[IR[25:21]]$; $B \leq Reg[IR[20:16]]$ $ALUOut \leq PC + (sign-extended(IR[15:0]) \ll 2)$			
执行 & 分支/跳转完成	$ALUOut \leq A \text{ op } B$	$ALUOut \leq A + sign-extend(IR[15:0])$	if (A==B) $PC \leq ALUOut$	$PC \leq \{PC[31:28], IR[25:0], 2'b00\}$
内存访问 & R型指令完成	$Reg[IR[15:11]] \leq ALUOut$	Load: $MDR \leq Memory[ALUOut]$ Store: $Memory[ALUOut] \leq B$		
内存		Load: $Reg[IR[20:16]] \leq MDR$		43

多周期功能描述： R型指令-1

ALUSrcA=1
ALUSrcB=00
ALUOp=10

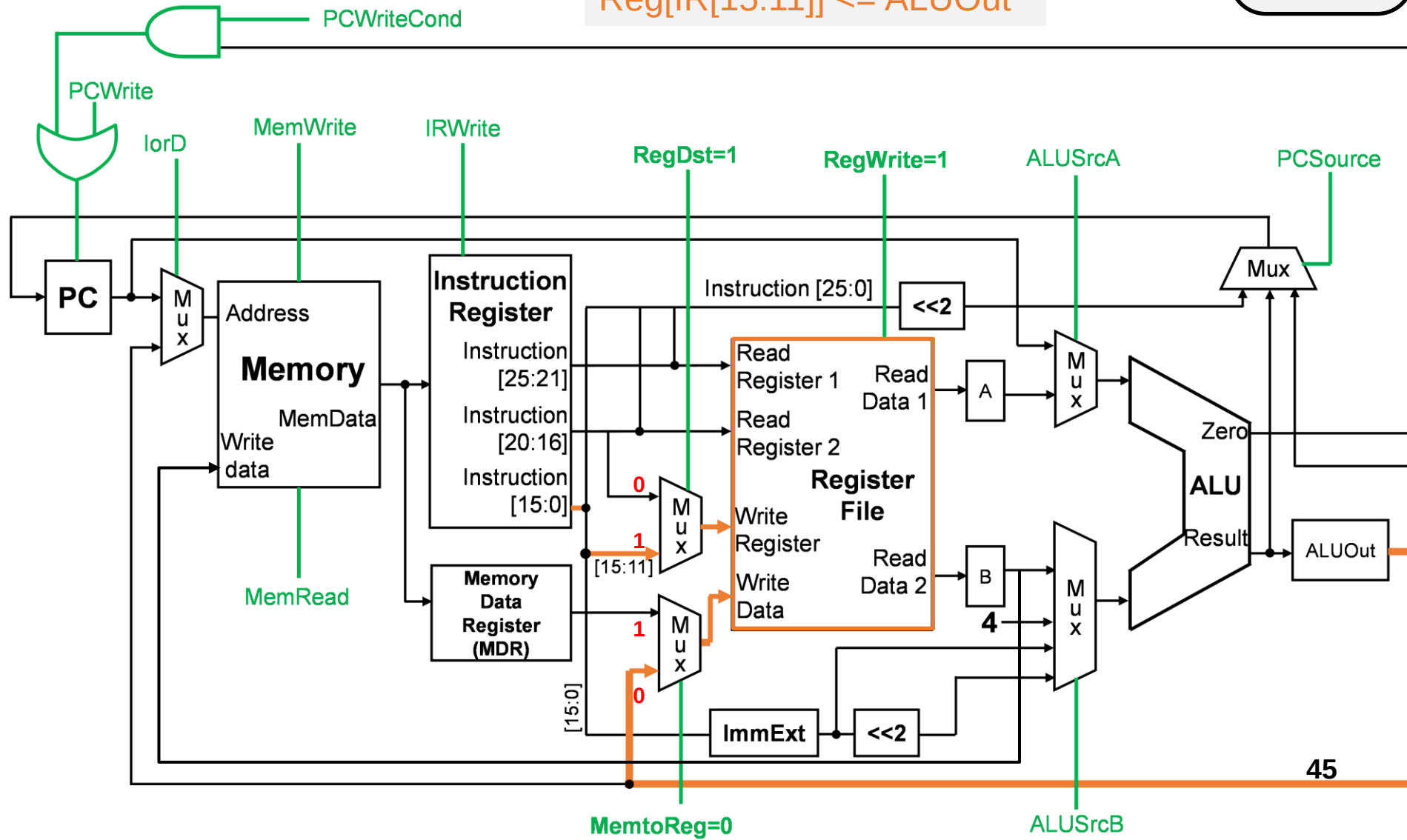
$ALUOut \leftarrow A \text{ op } B; \text{ op depends on IR}[5:0]$



多周期功能描述：R型指令-2

RegWrite
RegDst=1
MemtoReg=0

Reg[IR[15:11]] <= ALUOut



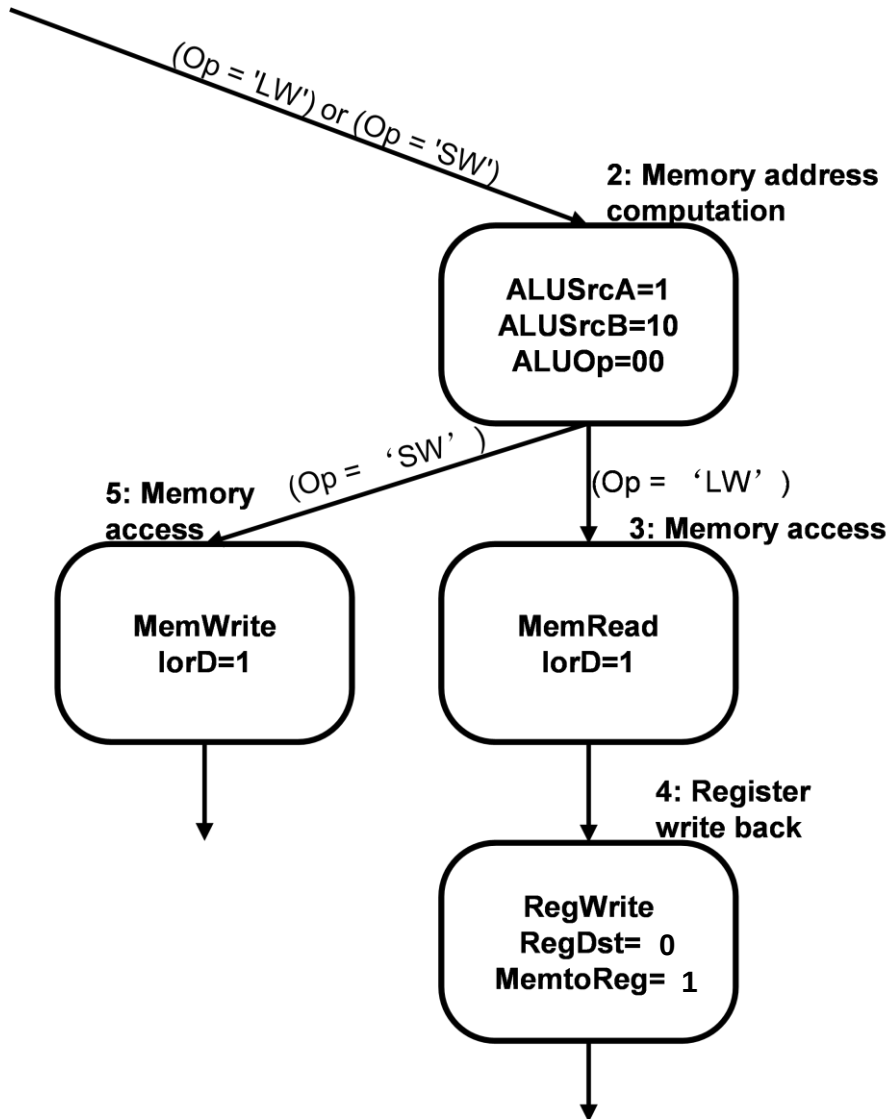
SW/LW指令的状态转移

- 有限状态机的视角

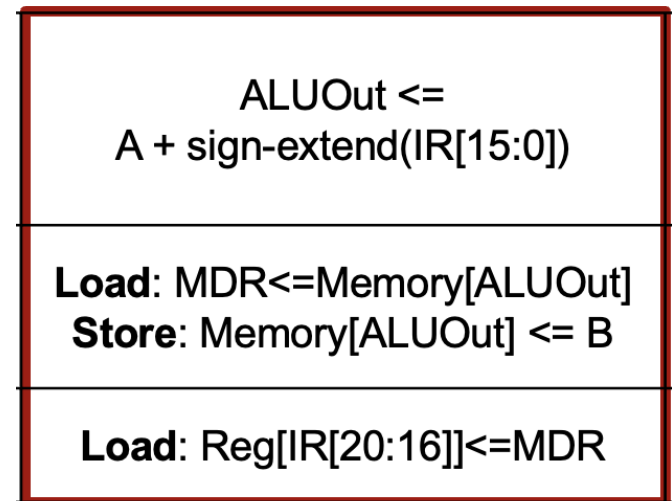
- 还有4个状态

步骤\指令类型	R型指令	内存访问指令	分支指令	跳转指令
取指令	IR <= Memory[PC] PC <= PC+4			
译码 & 寄存器堆读取	A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]] ALUOut <= PC + (sign-extended(IR[15:0]) << 2)			
执行 & 分支/跳转完成	ALUOut <= A op B	ALUOut <= A + sign-extend(IR[15:0])	if (A==B) PC<=ALUOut	PC <= {PC[31:28], IR[25:0], 2'b00}
内存访问 & R型指令完成	Reg[IR[15:11]] <= ALUOut	Load: MDR<=Memory[ALUOut] Store: Memory[ALUOut] <= B		
内存		Load: Reg[IR[20:16]]<=MDR		

SW/LW指令的状态转移



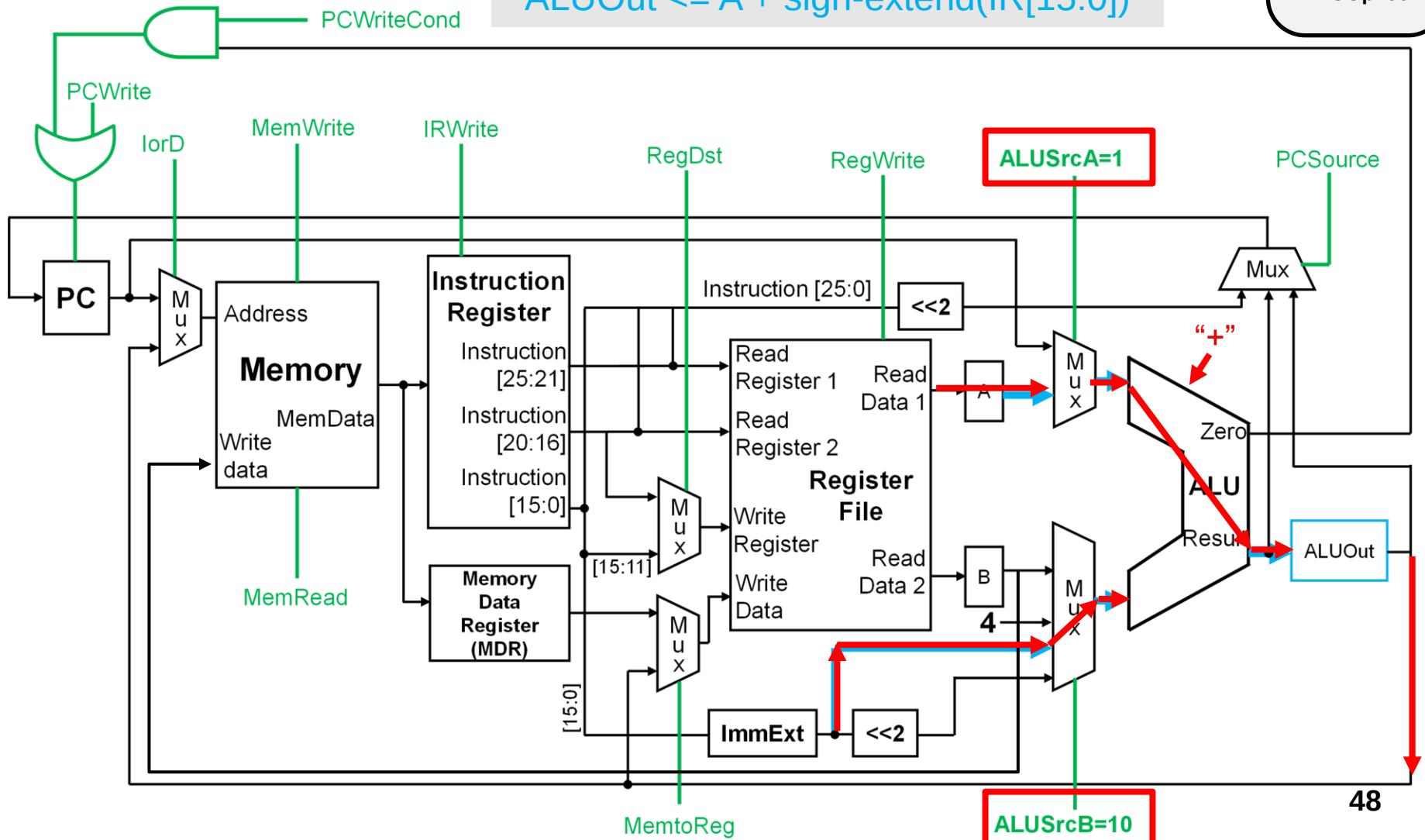
- 还有4个状态



多周期功能描述：访存地址计算

ALUSrcA=1
ALUSrcB=10
ALUOp=00

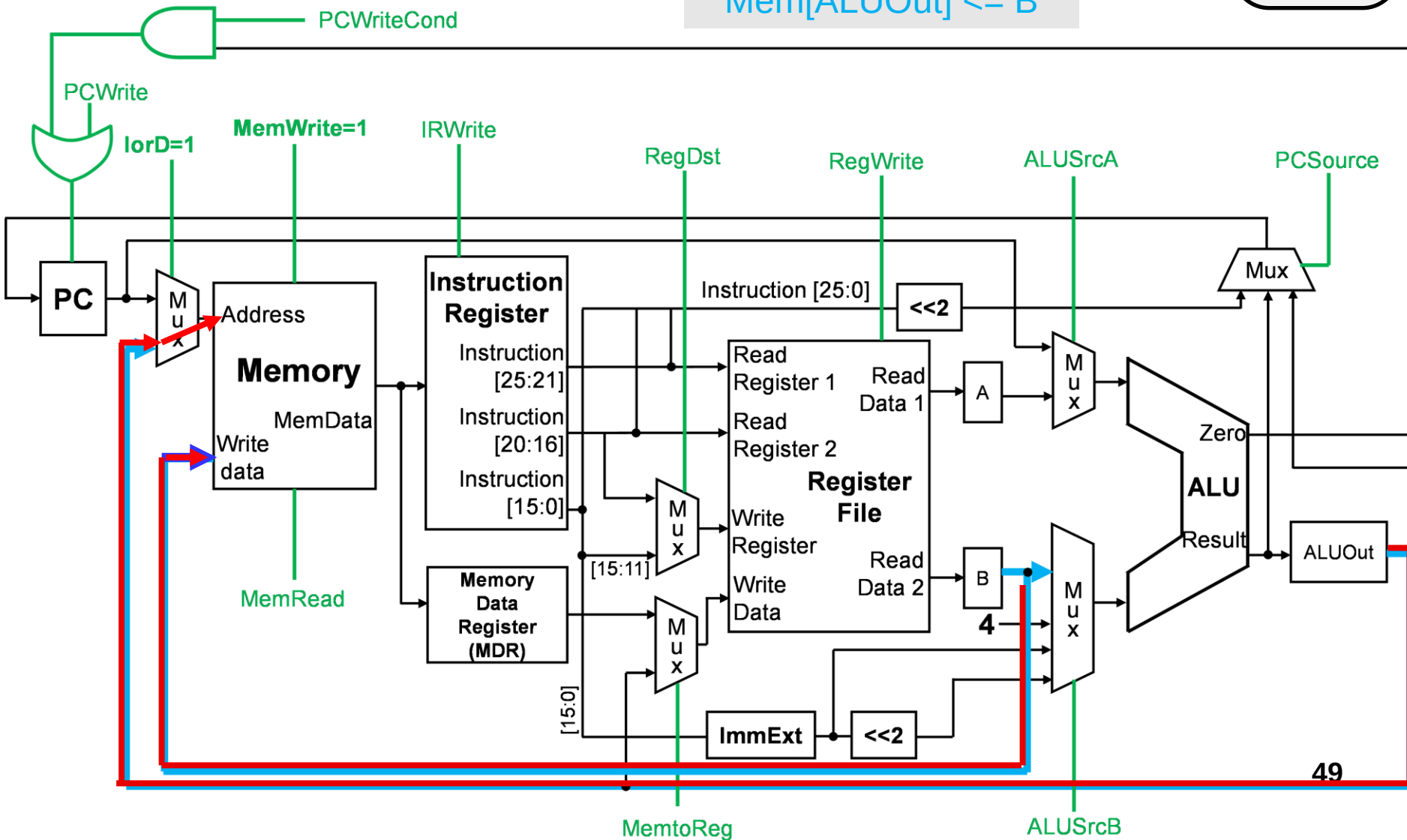
$$\text{ALUOut} \leq A + \text{sign-extend}(\text{IR}[15:0])$$



多周期功能描述：Store访存

MemWrite
lorD=1

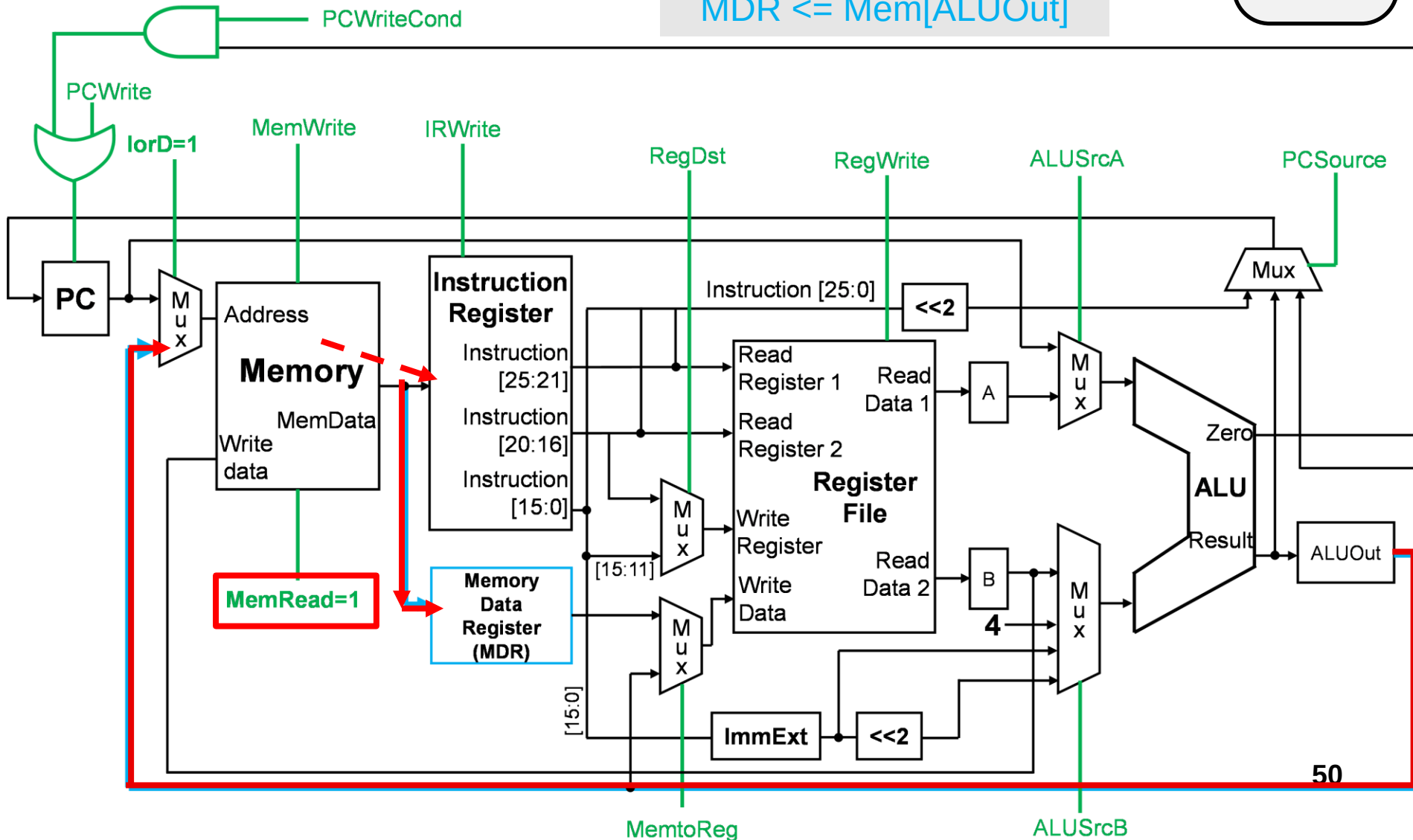
Mem[ALUOut] <= B



多周期功能描述：Load访存

MemRead
IorD=1

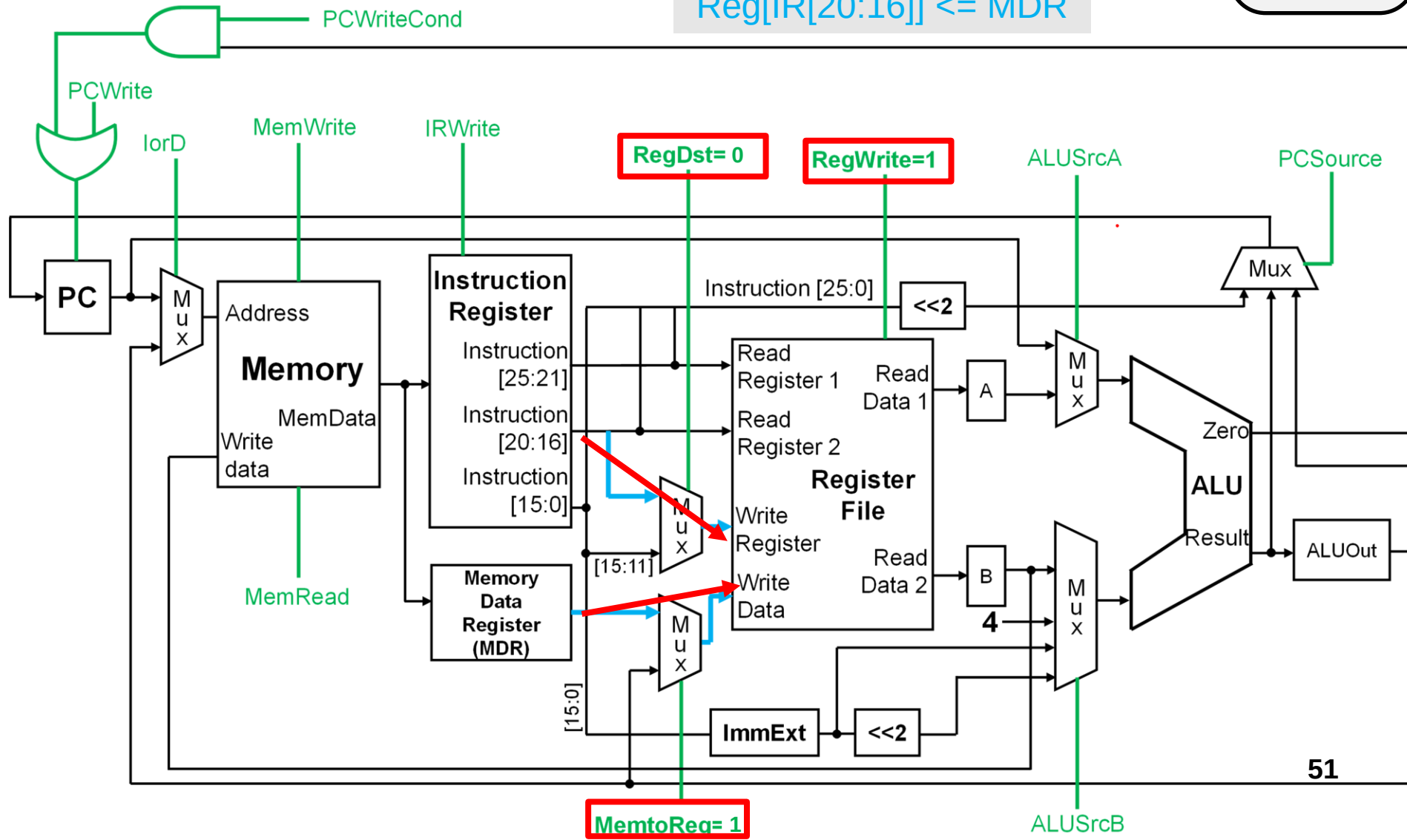
MDR <= Mem[ALUOut]



多周期功能描述：Load写回

RegWrite
RegDst=0
MemtoReg=1

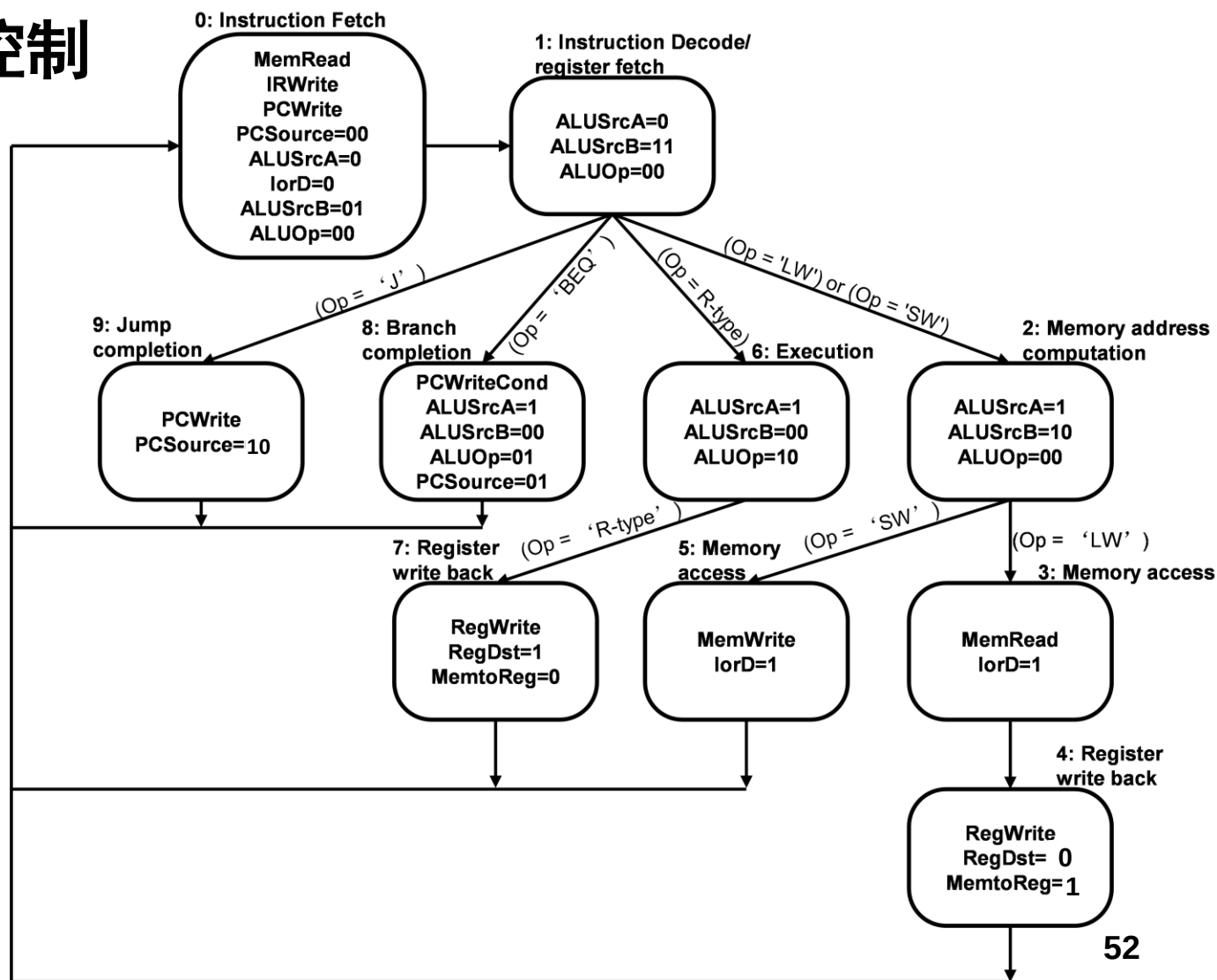
Reg[IR[20:16]] <= MDR



多周期数据通路

• 有限状态机控制

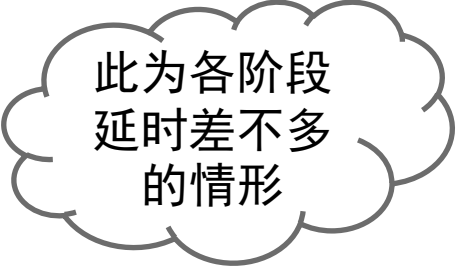
— 整体整合图



状态中没有显式表示的控制信号都是**无效的**，对于多路选择没有显式表示的都是**无关的**

多周期实现的性能（情景一）

- 假设主要功能单元的操作时间
 - 存储单元：200ps
 - ALU：150ps
 - 寄存器堆：150ps
 - 忽略其他延迟
- 指令执行各个步骤需要的时间
 - 步骤1：取指200ps
 - 步骤2：指令译码和读取寄存器150ps
 - 步骤3：指令执行，存储器地址计算或分支完成150ps
 - 步骤4：访存或R型指令写寄存器堆200ps
 - 步骤5：lw指令写寄存器堆150ps
- 多周期处理器的时钟周期：200ps
 - Note：单周期CPU时钟周期：200+150+150+200+150=850ps



此为各阶段
延时差不多的
情形

多周期实现的性能（情景一）

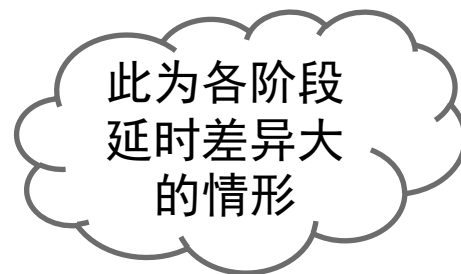
- 各指令执行需要的周期数和时间

- R型指令：4 cycles = 800ps
- lw指令：5 cycles = 1000ps
- sw指令：4 cycles = 800ps
- 分支指令：3 cycles = 600ps
- 跳转指令：3 cycles = 600ps

步骤\指令类型	R型指令	内存访问指令	分支指令	跳转指令
取指令	IR <= Memory[PC] PC <= PC+4			
译码 & 寄存器 堆读取	A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]] ALUOut <= PC + (sign-extended(IR[15:0]) << 2)			
执行 & 分支/跳转完成	ALUOut <= A op B	ALUOut <= A + sign-extend(IR[15:0])	if (A==B) PC<=ALUOut	PC <= {PC[31:28], IR[25:0], 2'b00}
内存访问 & R型指令完成	Reg[IR[15:11]] <= ALUOut	Load: MDR<=Memory[ALUOut] Store: Memory[ALUOut] <= B		54
内存		Load: Reg[IR[20:16]]<=MDR		

多周期实现的性能（情景二）

- 若主存储器相对更慢，逻辑操作更快呢？假设所需时间为
 - 存储单元：200ps；ALU：50ps；寄存器堆：50ps
 - 忽略其他延迟
- 指令执行各个步骤需要的时间
 - 步骤1：取指200ps
 - 步骤2：指令译码和读取寄存器50ps
 - 步骤3：指令执行，存储器地址计算或分支完成50ps
 - 步骤4：访存或R型指令写寄存器堆200ps
 - 步骤5：lw指令写寄存器堆50ps
- 单周期CPU周期550ps，小于760ps！
- 为什么？



多周期实现的性能（情景二）

- 若主存储器相对更慢，逻辑操作更快呢？假设所需时间为
 - 存储单元：200ps；ALU：50ps；寄存器堆：50ps
 - 存储器阶段太慢，拖累了原本较快的ALU计算等环节
 - 对于需要全部五个阶段的lw指令，比单周期低效
 - 只有当其他指令占比足够多，才能减少平均执行时间

单周期

Instruction fetch	Reg	ALU	Data access	Reg
----------------------	-----	-----	-------------	-----

多周期lw

Instruction fetch	Reg	ALU	Data access	Reg
----------------------	-----	-----	-------------	-----

多周期R型

Instruction fetch	Reg	ALU	Reg
----------------------	-----	-----	-----

多周期跳转

Instruction fetch	Reg	jump
----------------------	-----	------

这种指令必须足够多
整体才有优化

多周期不一定更有效！多周期的意义？

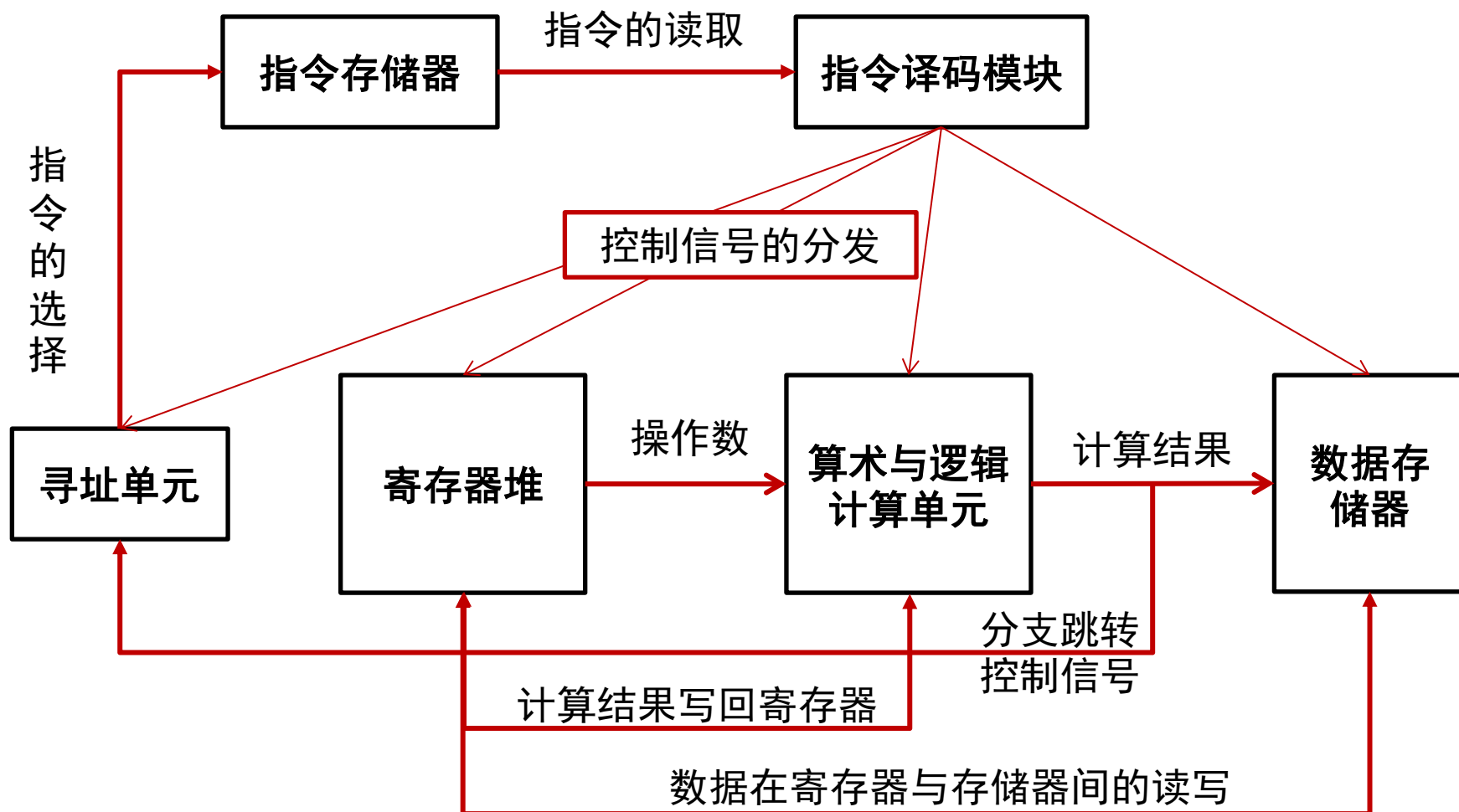
小结：多周期数据通路

- 允许指令占用不同数目的时钟周期
 - 避免慢指令拖累的程序执行速度
- 可在一条指令内多个步骤使用同一部件
 - 可减少硬件成本
- 性能改进取决于具体个步骤的延时
 - 与具体应用场景相关
- 可进一步改进，哪些改进？
 - 关键步骤用更多周期数执行
 - 片上缓存

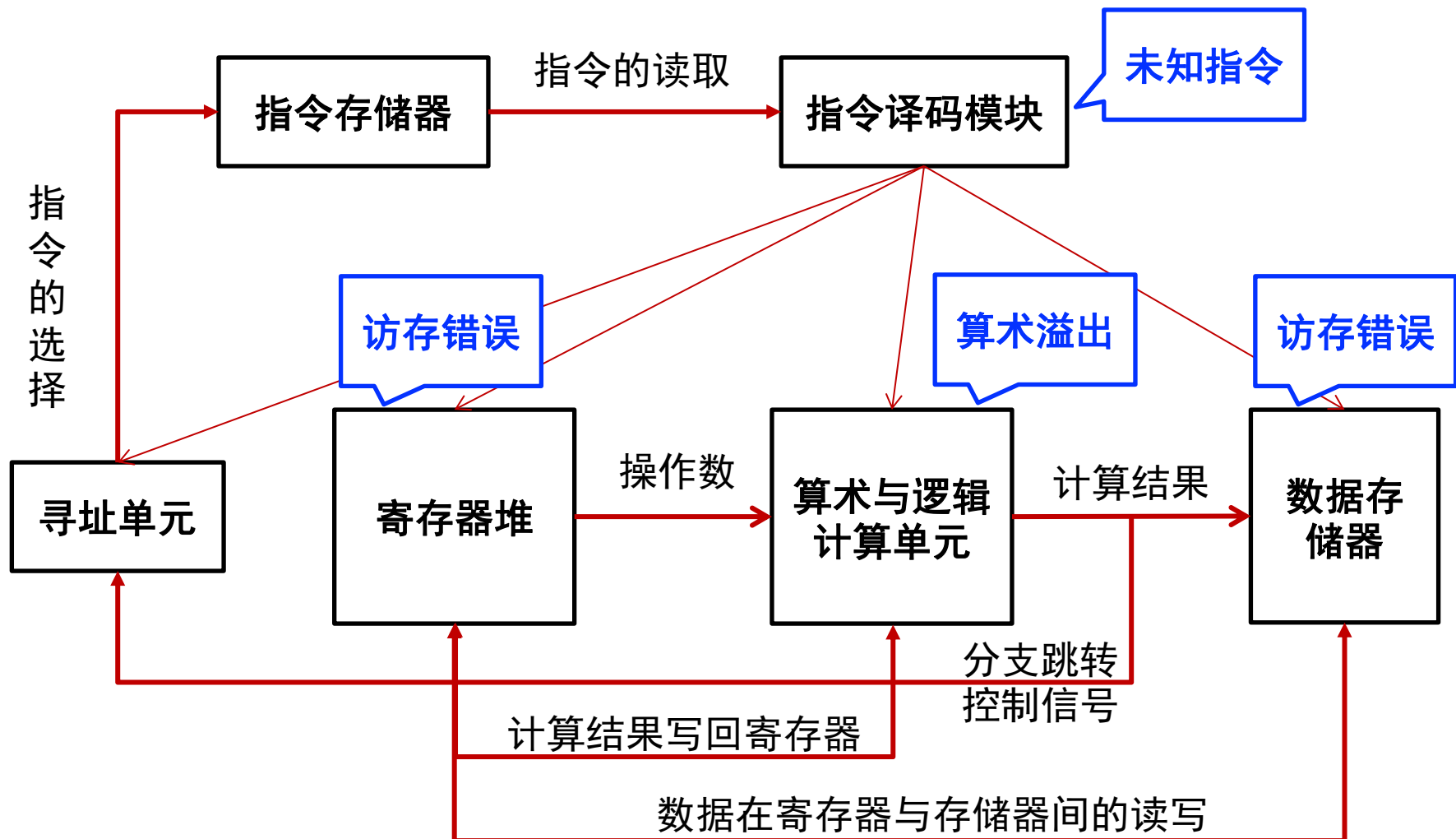
本节课目录

- 复习与回顾
- 单周期数据通路设计流程
 - 指令集和设计需求分析
 - 功能模块设计
 - 数据通路模块组装
 - 控制信号分析与逻辑设计
- 单周期数据通路延时分析
- 多周期数据通路
- 异常与中断

典型的处理器架构



MIPS处理器可能的异常类型



MIPS的异常和中断处理

- 一般来说，异常和中断是可以改变正常指令执行流程的事件 (除分支和跳转外)
- 异常
 - 内部不可预测事件、如溢出
- 中断
 - 外部不可预知事件、如I/O

Type of event	From where?	MIPS terminology
I/O device request	External	Interrupt
Invoke the operating system from user program	Internal	Exception
Arithmetic overflow	Internal	Exception
Using an undefined instruction	Internal	Exception
Hardware malfunctions	Either	Exception or interrupt

MIPS的异常和中断处理

- 异常和中断的区别

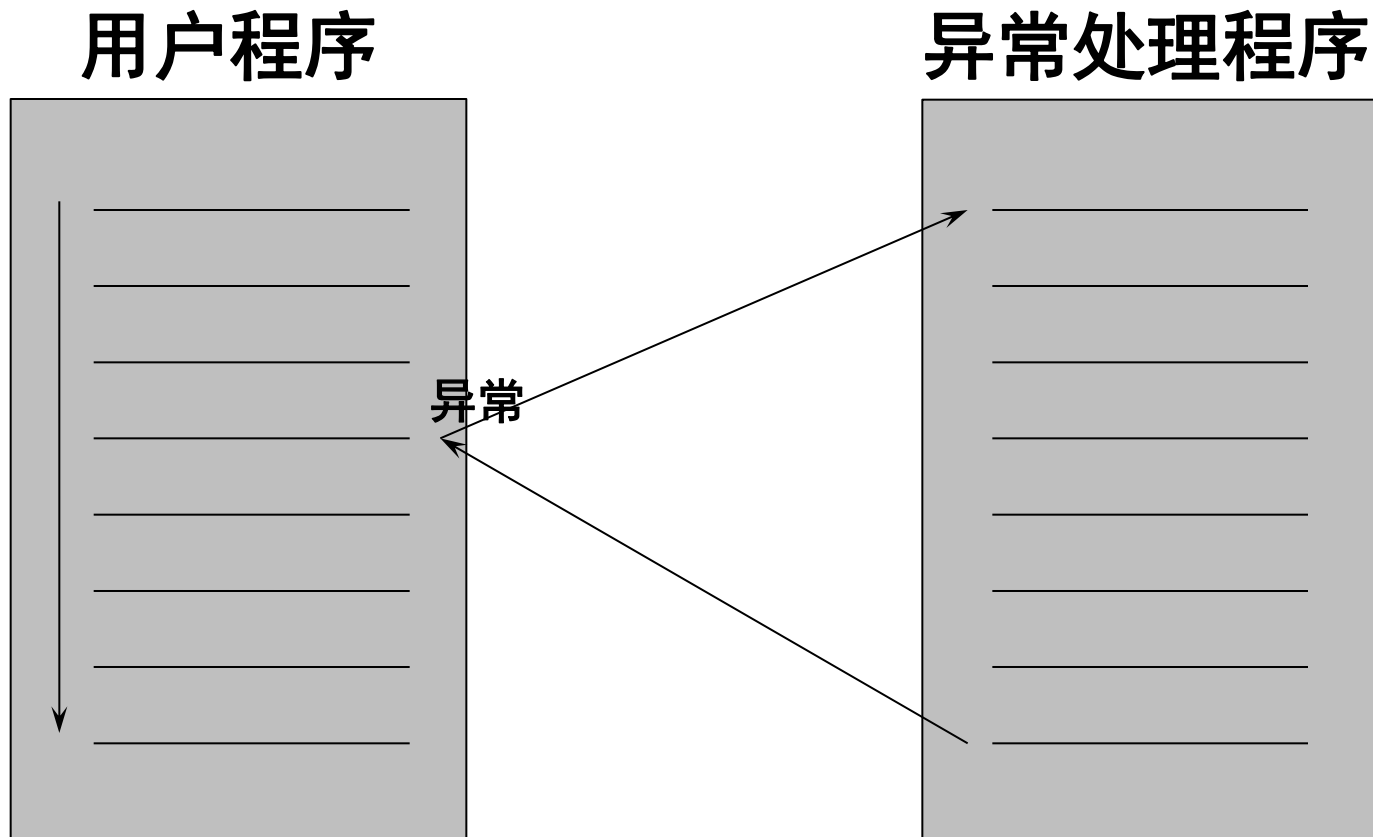
- 中断

- 异步事件
 - 与处理器正在执行的指令无关。
 - 主要由I/O设备产生
 - 一种特殊的异常

- 异常

- 一般为同步事件
 - 某一特定指令执行的结果。
 - 在相同条件下，异常可以重现。
 - 例如，内存访问错误、被零除等是典型的异常
 - 系统服务调用通常视作异常

异常处理



异常处理

- **正常控制流**
 - 顺序执行、分支、跳转、过程调用/返回
- **发生异常时，要做什么？**
 - 系统要实施一定动作来处理异常
 - 必须记录下来产生异常的指令的地址
 - 必须保存和恢复用户程序状态
 - 异常处理结束，将控制交还给用户程序

异常处理程序的寻址（向量方式）

- 获得异常处理程序入口地址的方法
 - **向量方式**：每个异常都有自己的异常处理程序，其入口保存在一个固定的地址处
 - X86采用向量方式，例如：

Exception type	Exception vector address (in hex)
Undefined instruction	8000 0000 _{hex}
Arithmetic overflow	8000 0180 _{hex}

- 操作系统通过异常向量地址得知异常原因

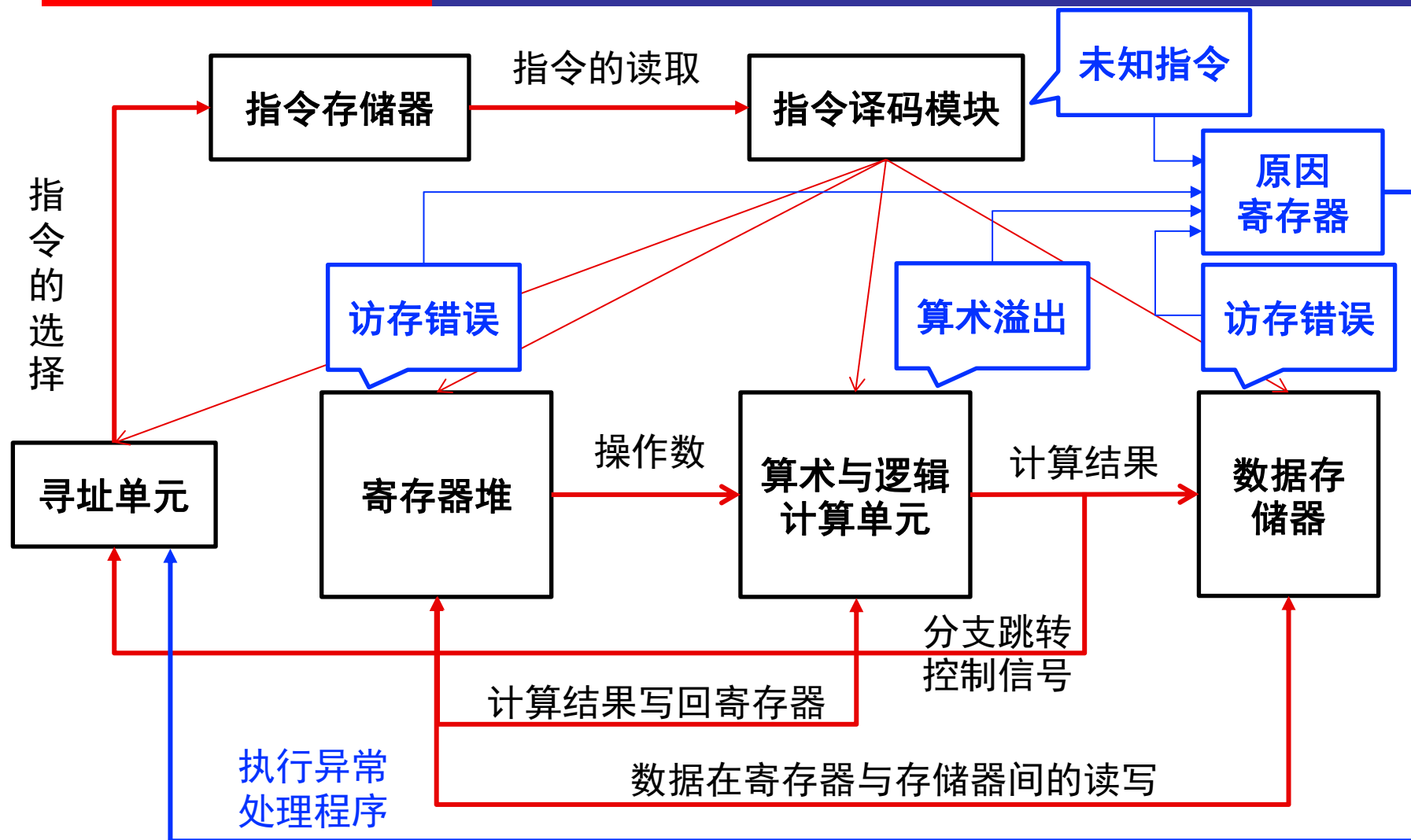
异常处理程序的寻址（原因寄存器）

- 获得异常处理程序入口地址的方法
 - 原因寄存器 (Cause Register)
 - 只使用一个通用的异常处理程序
 - 每个异常事件必须将足够的信息加载到原因寄存器中，以便异常处理程序知道是何种异常发生，并采取相应的措施
 - MIPS采用原因寄存器方式
 - 所有异常处理程序入口相同
 - 操作系统根据原因寄存器确定异常原因
 - 中断是一种特殊的异常事件
 - ExcCode=0 原因寄存器

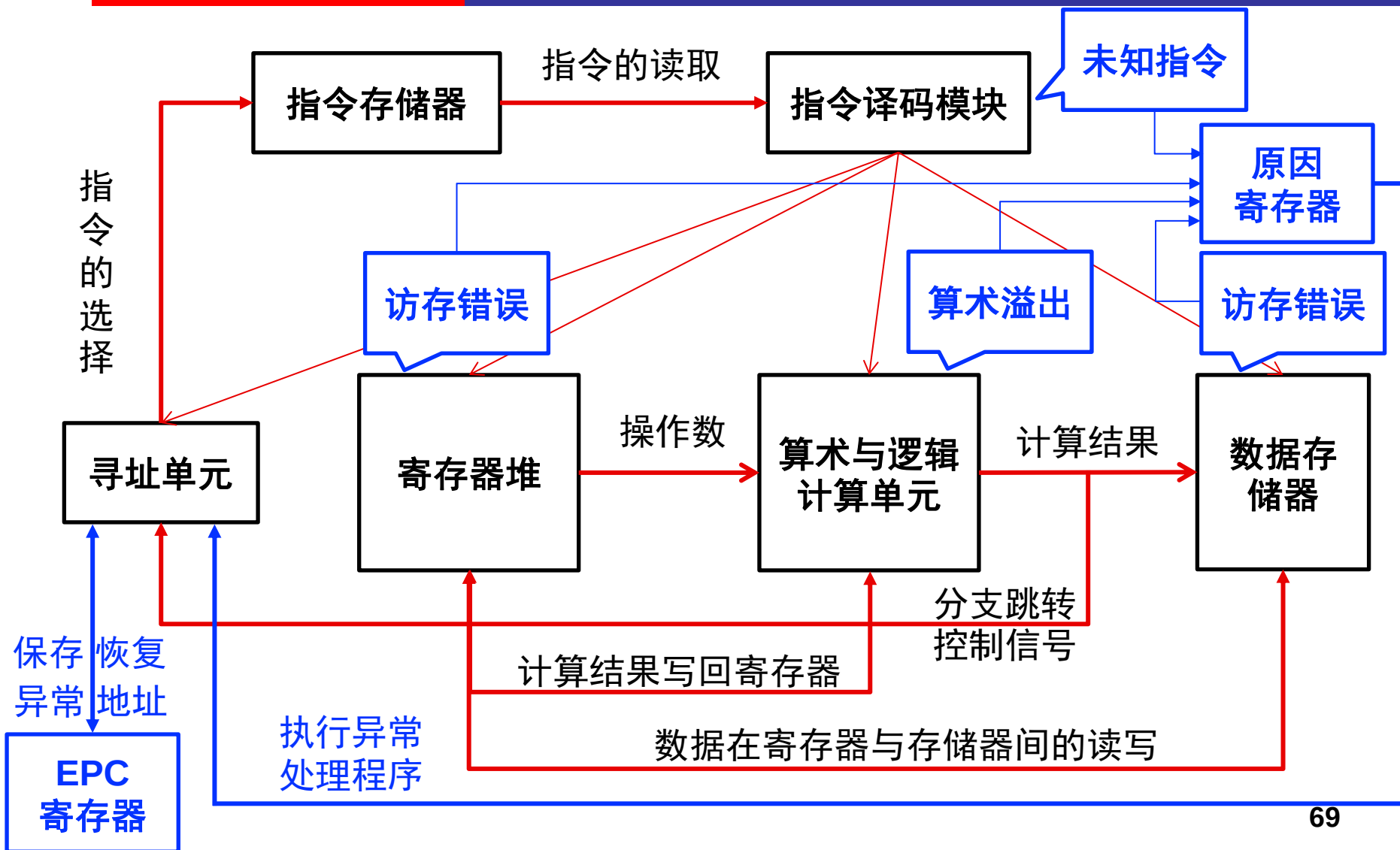
MIPS的异常处理

- EPC寄存器(exception program counter)
 - 用于保存造成异常的指令地址
 - 异常处理完后可以将PC返回该地址继续执行
 - 向量中断同样需要EPC寄存器

记录异常原因



保存/恢复异常指令地址



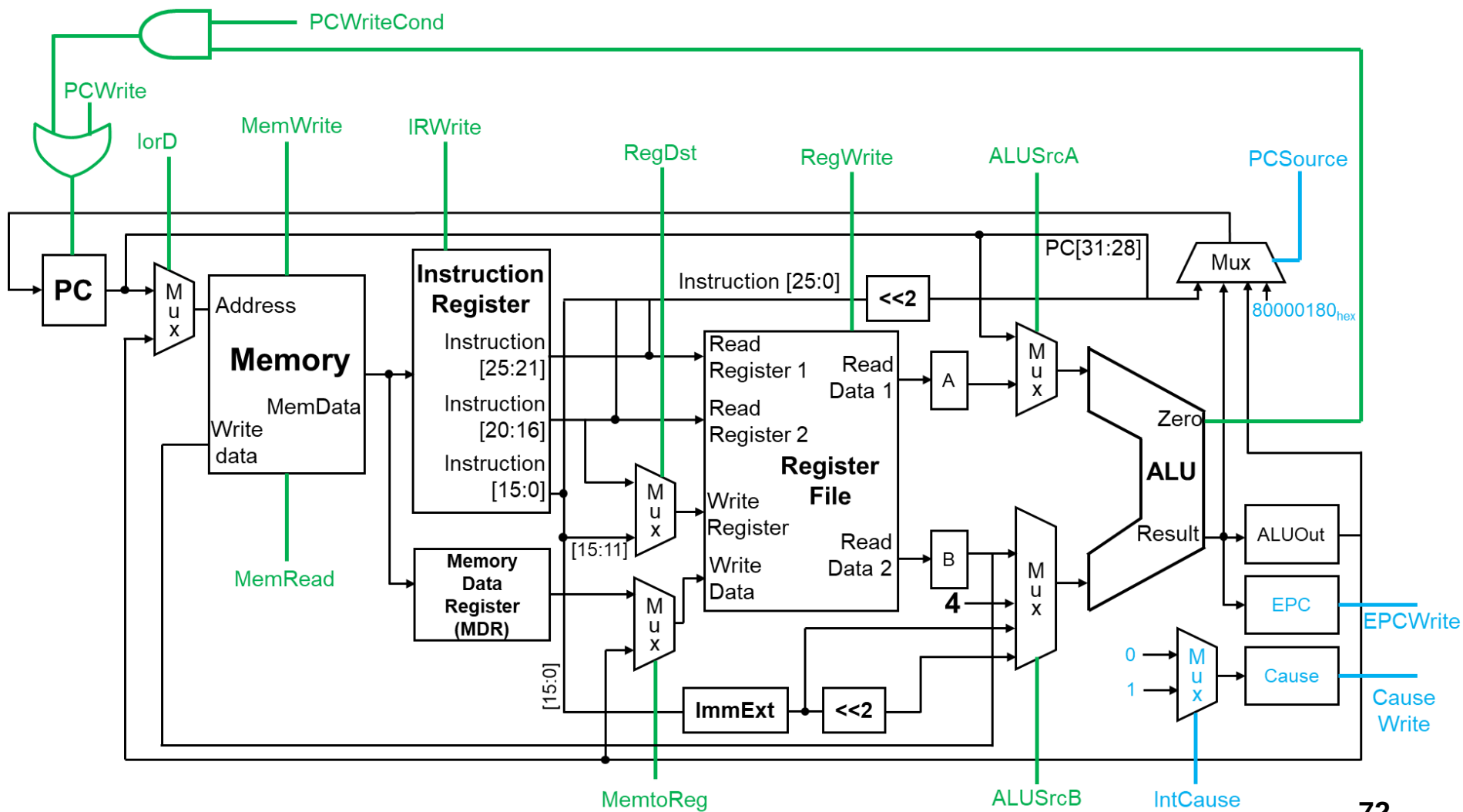
实现简化MIPS的异常处理

- 假设要处理两种异常
 - 未定义指令的执行
 - 算术溢出
 - 异常处理程序的入口假设为80000180_{hex}
- 数据通路的修改：
 - 增加**EPC寄存器**——32位寄存器，用于保存受影响的指令的地址
 - 增加**Cause寄存器**——用于记录产生异常事件原因的寄存器
 - 32位MIPS中一些位目前还没有定义用途
 - 假设其最低位对两种异常原因进行了编码：未定义指令=0，算术溢出=1，后续探索数据通路实现方法

实现简化MIPS的异常处理

- 数据通路的修改：
 - 增加PC的源：异常处理程序的入口地址80000180_{hex}
 - 由于在每条指令执行的第一个周期(取指令)将PC+4写入了PC，所以写入EPC的地址应该是PC-4。
 - 需要增加一个多路选择器
 - 0: 未定义指令
 - 1: 算术溢出
 - 所需的控制信号：
 - EPCWrite: EPC写入使能
 - CauseWrite: Cause写入使能
 - PCSource: 2位, PC源选择信号
 - IntCause: 1位, 异常原因选择信号

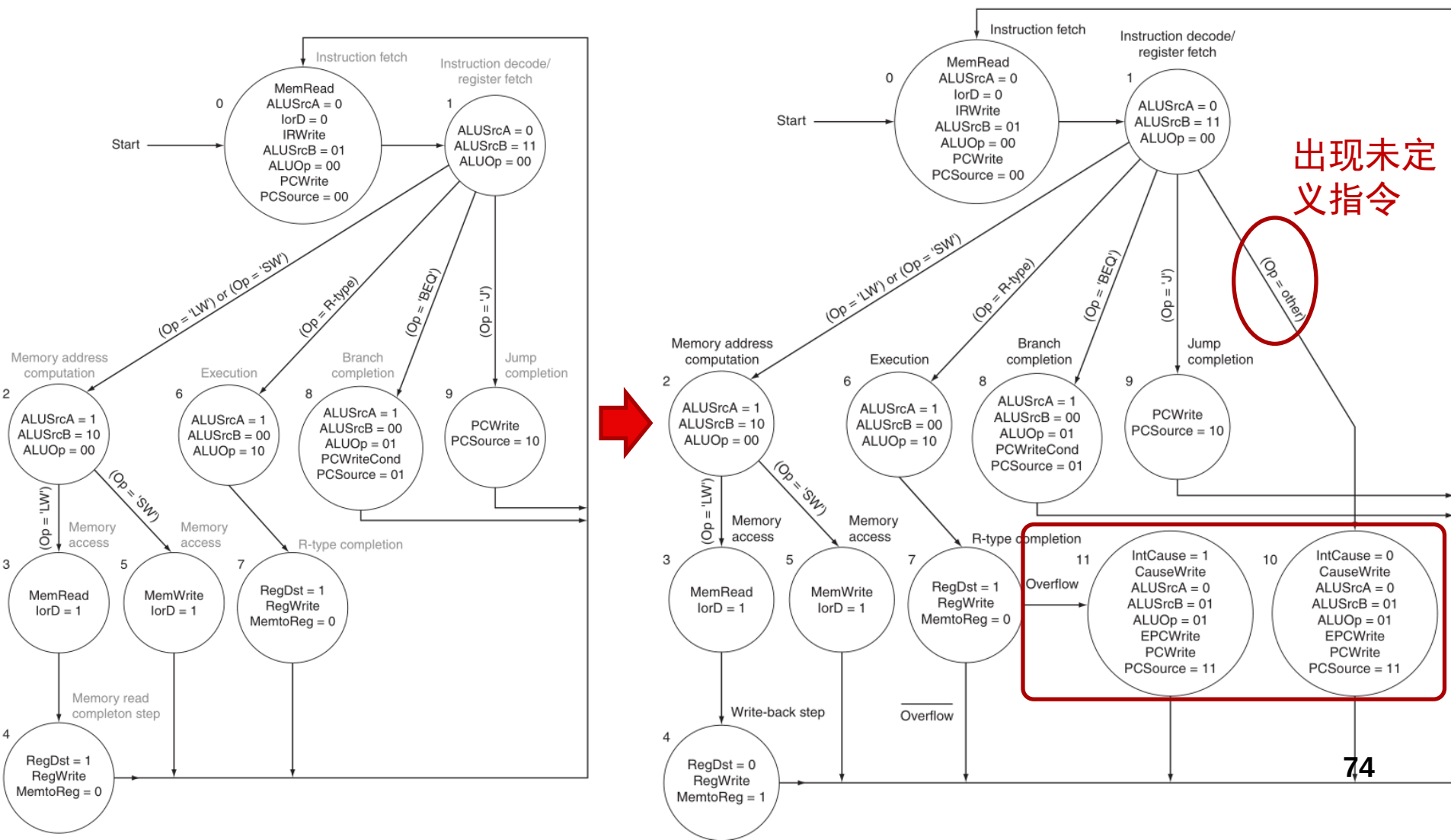
实现简化MIPS的异常处理



控制单元如何检测异常

- 未定义指令
 - 对于特定的操作码数值，如果状态1没有后续状态，那么就检测到了未定义指令
 - 为处理这一异常，定义一个新的后续状态(10)来接受所有未定义的操作码，用Other来指明这些未定义的操作码
- 算术溢出
 - ALU对于算术溢出能够提供一个称为Overflow的信号
 - 在修改后的FSM中将使用这一信号来指定另外一个可能的后续状态(11)

加入异常检测后的FSM状态图



本节课小结

- 多周期处理器

- 不同指令执行需要不同的步骤，有不同的关键路径
- 多周期思路：每条指令用多个周期完成，不同指令可以占用不同的时钟周期数
- 数据通路的修改
- 多周期的状态机描述

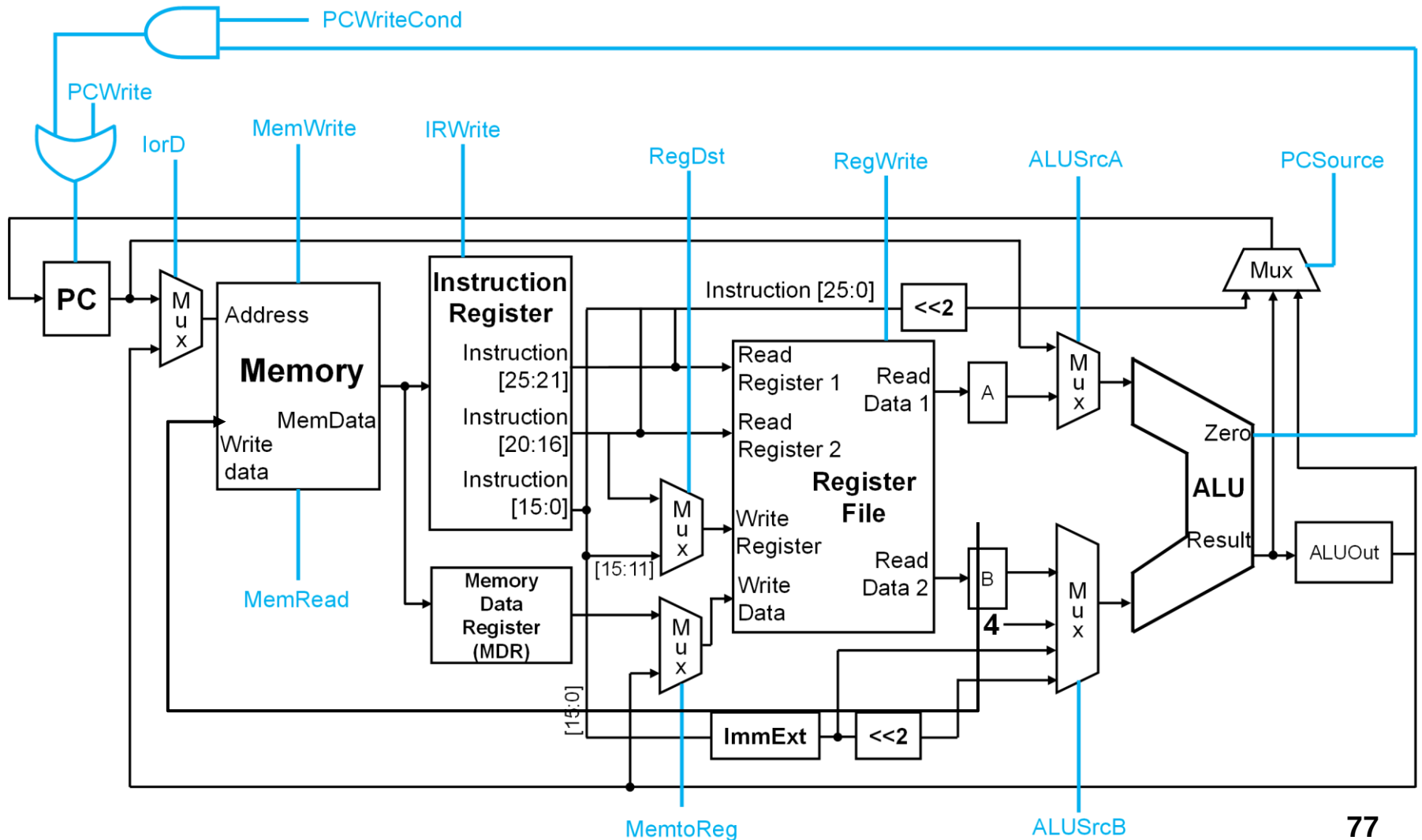
- 异常和中断

- 异常和中断的定义与区别
- MIPS异常处理的方式

作业

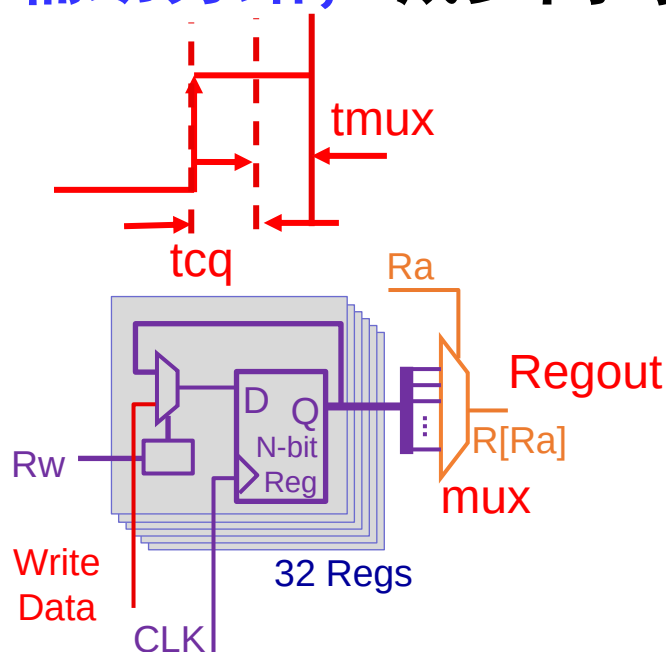
- 要求
 - 线上交电子版
 - 不要抄袭，迟交一周内扣50%，答案公布后再交不给分
- 本次作业
 - 网络学堂作业区

附：多周期数据通路

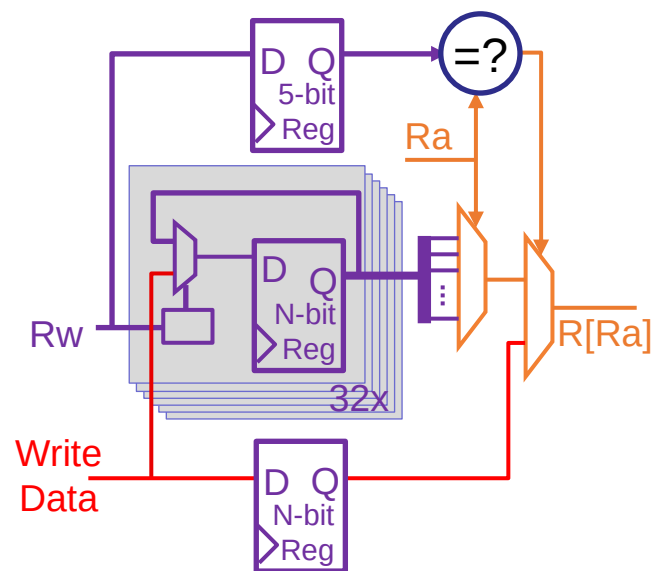


附：寄存器堆访问

- 时钟触发边沿写入，且支持“先写后读”
 - 写入到RF的数据在同周期内可以直接被访问
 - RF/Memory阵列大，写入的更新数据需建立时间，如50/200ps
- 辅助旁路，减少同寄存器“先更新再读出”的延时



先写后读



辅助旁路读

附：单周期实现的性能

• 各类指令的执行时间细分

- 若不同指令可在不同长度的一个时钟周期内执行完，求该程序的指令平均执行时间？

Lw: 25%
Sw: 10%
ALU: 45%
Beq: 15%
J: 5%

$$600 \times 25\% + 550 \times 10\% + 400 \times 45\% + 350 \times 15\% + 200 \times 5\% = 447.5ps < 600ps!$$

指令类型	指令存储器	读寄存器堆	ALU运算	数据存储器	写寄存器堆	总延时
R型	200ps	50ps	100ps	0	50ps	400ps
Load类型	200ps	50ps	100ps	200ps	50ps	600ps
Store类型	200ps	50ps	100ps	200ps	0	550ps
Branch	200ps	50ps	100ps	0	0	350ps
Jump	200ps	0	0	0	0	200ps